

Series Q Q2

Hands-on I: the single-sample standard pipeline from raw matrix to cell annotation

10x's public GBM 5k, 5,604 cells. From loading to named clusters and state scores for malignant cells.

About 55 min | Prerequisites: Q1 | Scripts: R/00-03 | Deeper: B1, B5-B10

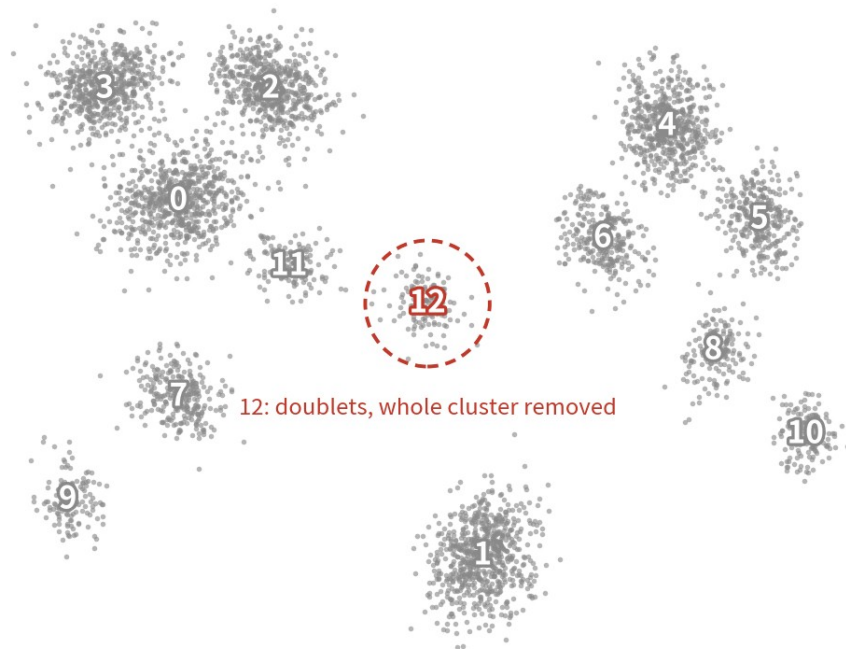


[https://github.com/Charlene717/
scRNA-seq-tutorial-Qseries](https://github.com/Charlene717/scRNA-seq-tutorial-Qseries)

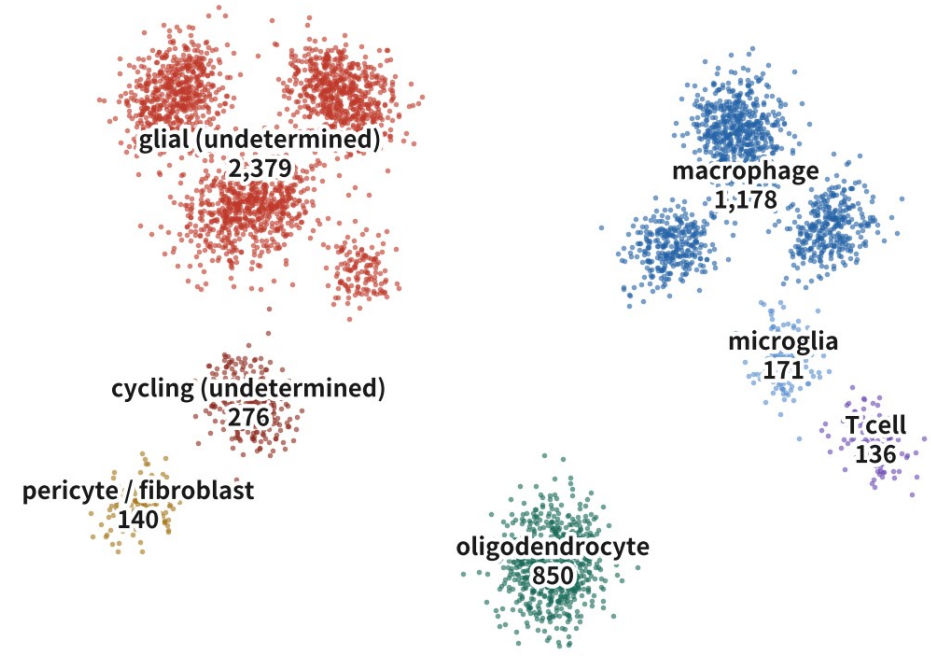
The end result of this episode

This episode's endpoint: from numbers to names

after clustering: 13 numbers



after annotation: 7 names



13 clusters is not 13 cell types: four are glial, three are macrophage, and one is a doublet cluster removed whole. Numbers reshuffle with any parameter; only names go into a paper

13 numbers, 7 names: four clusters are glial, three are macrophage, one doublet cluster is removed whole. Malignant cells are labelled 'glial (undetermined)', not declared tumour

THE RULE FOR THIS EPISODE

**Every code block answers two questions:
what this line does, and why the
parameter is set that way.**

Full code lives in three practice scripts (R/01-03), each ending with exercises. 'Deeper: Bx' points to the Series B episode that goes all the way.

Episode goals

1

Load GBM 5k into Seurat v5, set three QC cutoffs from this dataset's distributions, flag doublets

2

Run normalization → HVG → scaling → PCA → clustering → UMAP with three sanity checks (PC1, nPC, per-cluster QC)

3

Draw the four major groups with gate genes, name clusters with markers + SingleR, score malignant states with Neftel gene sets

01

Environment and data

Script 00: install, download, set up the project

Practice data: a real tumour sample (GBM 5k)

Practice data: Human Glioblastoma Multiforme 5k

source	10x Genomics public dataset (Chromium 3' v3)
tissue	dissociated human glioblastoma tumour
cells	5,604 (called by Cell Ranger 4.0.0)
files	filtered_feature_bc_matrix.tar.gz ($\approx$ 30 MB)
why	Malignant, immune, vascular and normal brain; 5 min on a laptop

What it cannot answer

- ✗ One patient only: no patient-level inference ($n=1$)
- ✗ No matched normal: malignant calling relies on internal references (immune, oligodendrocytes)
- ✗ No clinical data: cannot link to response or survival
- Q3 switches to a 4-patient dataset to fill these gaps

Read the right side first: what a dataset can and cannot answer matters more than its cell count

Install, pin the version, set the seed

```
install.packages(c("Seurat", "dplyr", "ggplot2", "clustree"))
BiocManager::install(c("SingleR", "cellx", "scDbFinder"))
remotes::install_github("chris-mcginnis-ucsf/DoubletFinder") # GitHub only
library(Seurat); library(dplyr); library(ggplot2)
stopifnot(packageVersion("Seurat") >= "5.0.0") # v4 tutorial code breaks on v5
set.seed(1234) # fixed for the whole series
for (d in c("data", "R", "output")) dir.create(d, showWarnings = FALSE)
```

No setwd(), no absolute paths: open an RStudio Project and the root is the working directory | Script 00 | Deeper: B1

Download, extract, load

```
url<-paste0("https://cf.10xgenomics.com/samples/cell-exp/4.0.0/",
            "Parent_SC3v3_Human_Globlastoma/",
            "Parent_SC3v3_Human_Globlastoma_filtered_",
            "feature_bc_matrix.tar.gz")
download.file(url, "data/gbm5k.tar.gz", mode = "wb") # about 30 MB
untar("data/gbm5k.tar.gz", exdir = "data/gbm5k")

mtx.dir<- "data/gbm5k/filtered_feature_bc_matrix"
gbm.data<-Read10X(data.dir=mtx.dir) # takes the folder
gbm<-CreateSeuratObject(counts = gbm.data, project = "gbm5k",
                        min.cells = 3, min.features = 200)
gbm
```

An object of class Seurat

~ 25,000 features across ~ 5,500 samples within 1 assay

min.cells / min.features are a coarse screen; real cutoffs come from distributions next. Use the numbers we actually get | Script 01 §1

First checks after loading: dimensions, sparsity, gene names

```
din (gbm )           # genes x cells
cnt<-LayerData(gbm , layer= "counts")
1 -Matrix::nnzero(cnt)/prod(din (cnt)) # sparsity:> 0.85 is normal
summary(Matrix::colSums(cnt))          # totalUMIs per cell
grep("^MT-", rownames(gbm ), value = TRUE) # human MT-, mouse mt-
head(grep("^RP[LS]", rownames(gbm ), value = TRUE)) # ribosomal
cnt[intersect(c("PTPRC ", "SOX2", "MBP"), rownames(cnt)),1:5] # three lineage marker genes
```

```
[1] 25xxx 55xx
[1] 0.86
  Min. 1st Qu. Median   Mean 3rd Qu.  Max.
  5xx   2xxx   4xxx   6xxx   8xxx  6xxxx
[1] "MT-ND1" "MT-ND2" "MT-CO1" ... (13 genes)
```

Thirty seconds that prevent three accidents: wrong species (MT- vs mt-), loading the raw matrix (hundreds of thousands of columns), Ensembl IDs instead of symbols | Script 01 §1

raw, filtered, and ambient RNA

Cell Ranger hands us two matrices; we load filtered, but raw has one more use

raw matrix



`raw_feature_bc_matrix`:
every Barcode (hundreds of thousands), mostly empty droplets. The RNA in empty droplets is a sample of the background 'soup'

filtered matrix



`filtered_feature_bc_matrix`:
Barcodes Cell Ranger calls cells via EmptyDrops (5,604 here). This is what we load; doublets and damaged cells are still inside

Ambient RNA



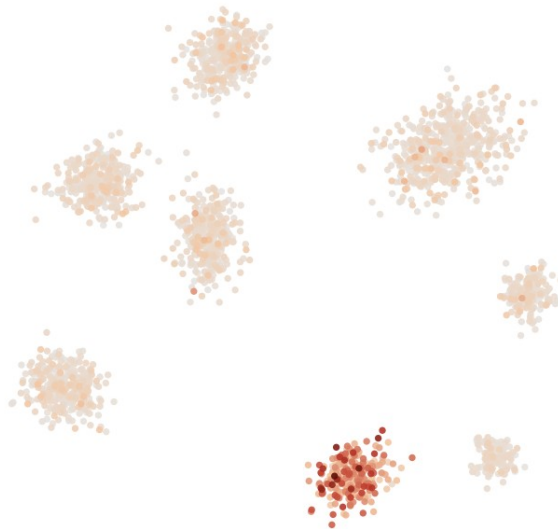
mRNA from lysed cells drifts into every droplet: each cell carries a little of everyone else. Worst in necrotic, slowly dissociated tumours

Ambient RNA: how to spot it, how to remove it

schematic / simulated

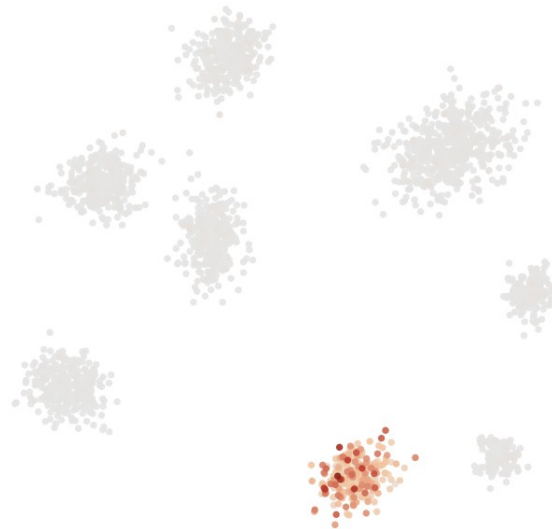
Ambient RNA: every cell in the matrix carries a little 'soup'

before: MBP



every cell carries a little MBP—not because oligodendrocytes are everywhere, but because mRNA from lysed cells drifts into every droplet

after SoupX: MBP



background estimated and subtracted; the marker lights only where it should

What SoupX does

- ① estimate the soup profile from empty droplets (low-UMI Barcodes in the raw matrix)
- ② estimate each cell's contamination fraction p from genes a cell type cannot express (e.g. MBP outside oligodendrocytes)
- ③ subtract $p \times \text{soup}$ from every cell, return corrected integer counts

When it is mandatory: necrotic, slowly dissociated tumours; $p > 10\%$; when gate genes (PTPRC, MBP, HBB) show a little everywhere

A gate gene showing a little everywhere → estimate contamination first; above $p \approx 10\%$, run SoupX (or CellBender) before QC

SoupX: estimating and removing ambient RNA (optional, script 01 \$0)

```
library(SoupX)
sc <- load10X("data/gbm_5k_outs/") # needs raw + filtered side by side
sc <- setClusters(sc, gbm$seurat_clusters) # a rough clustering first
sc <- autoEstCont(sc) # estimate the contamination fraction rho
sc$fit$rhoEst # e.g. 0.06 -> optional at this level
adj <- adjustCounts(sc, roundToInt = TRUE) # corrected integer counts
gbm <- CreateSeuratObject(adj, project = "gbm_5k",
                           min.cells = 3, min.features = 200)
```

Needs the raw matrix, so datasets that ship filtered-only cannot do this; always keep raw for our own data. Skip below $p \approx 5\%$ | Deeper: B5

Reference: Young MD, Behjati S. *Gigascience*. 2020;9(12):giaa151. doi:10.1093/gigascience/giaa151

02

QC: three cutoffs set from the data

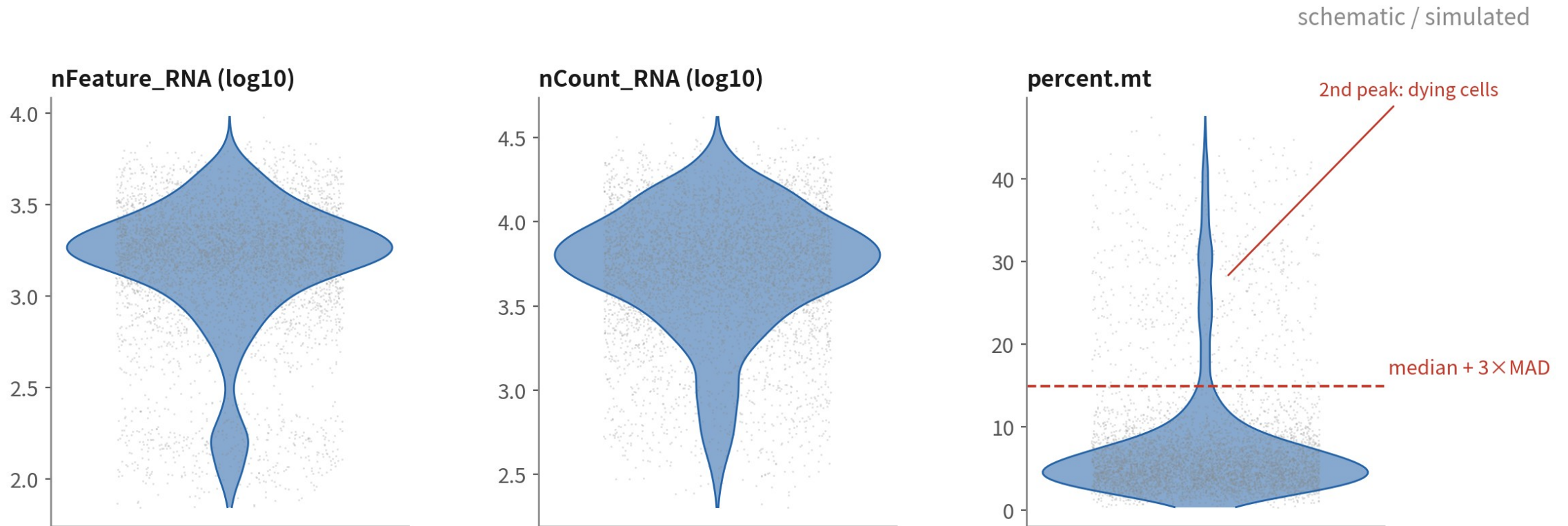
Look first, cut second; save the raw object before filtering

The three QC metrics: inspect the distributions before filtering

```
gbm[["percent.mt"]] <- PercentageFeatureSet(gbm, pattern = "^MT-")  
saveRDS(gbm, "output/rds/01_gbm_raw.rds") # save before filtering: a cut cannot be undone  
  
VlnPlot(gbm, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"),  
        ncol = 3, pt.size = 0.05)  
FeatureScatter(gbm, "nCount_RNA", "percent.mt")  
FeatureScatter(gbm, "nCount_RNA", "nFeature_RNA")
```

nCount catches empties and doublets, nFeature debris, percent.mt dying cells | Script 01 §2 | Deeper: B5

QC distributions: a tumour sample as the example



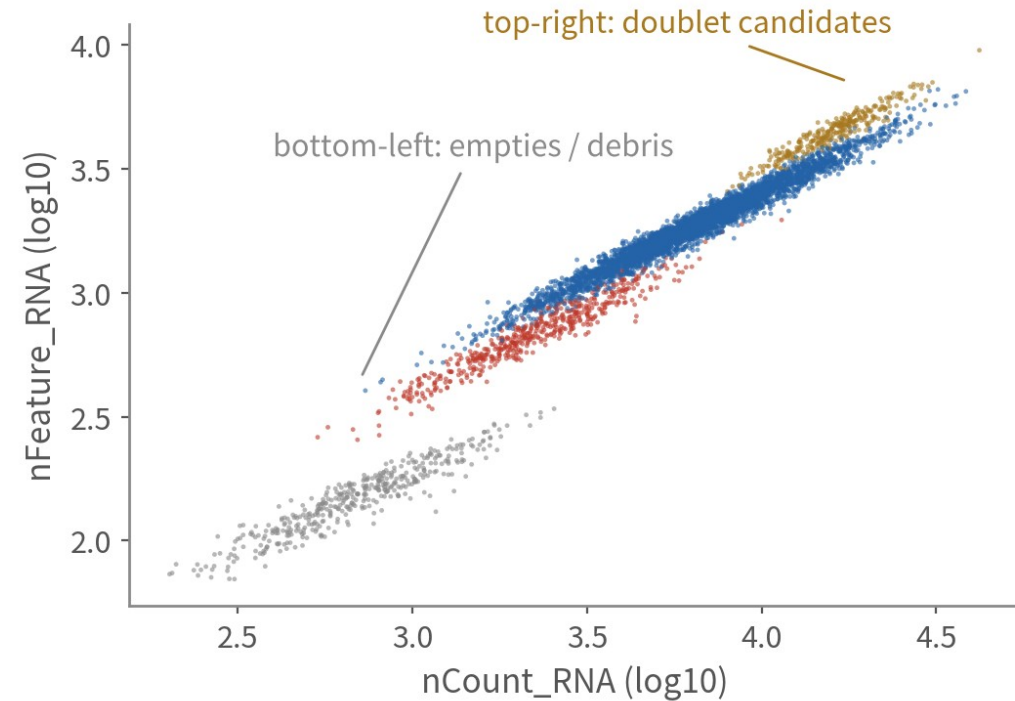
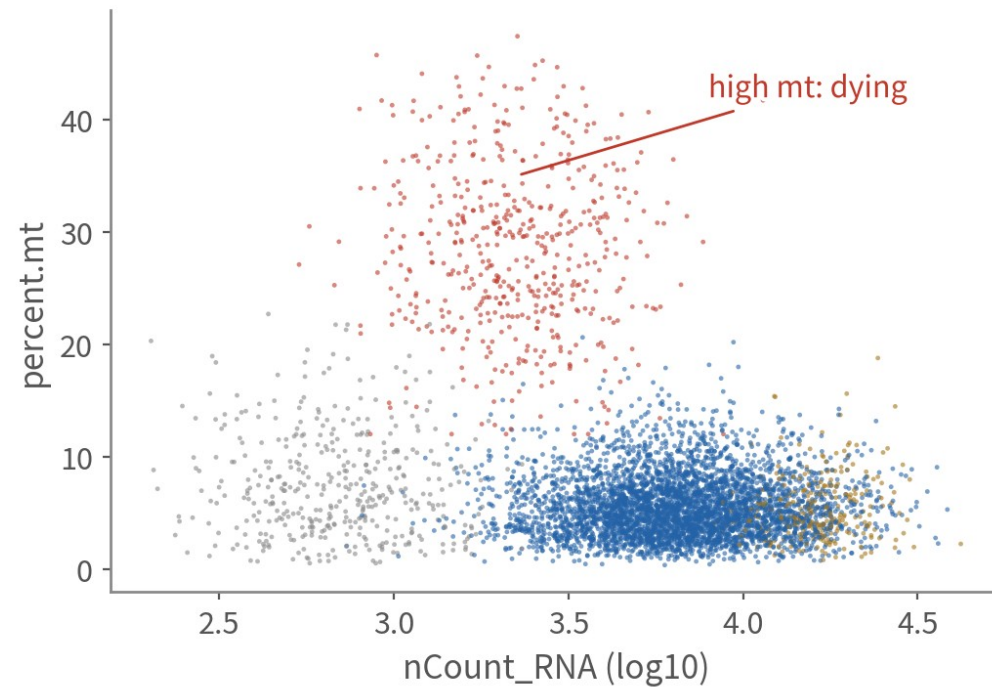
Tumour percent.mt runs higher than PBMC (malignant cells are metabolically active) and is often bimodal. A copied 5% cut removes a large slice of live cells

percent.mt runs higher than PBMC and is often bimodal. The second mode is dying cells; the line goes between the modes

QC scatter plots: where the three failure modes sit

Two scatters read together: each kind of junk owns a corner

schematic / simulated



One metric misleads; read them in pairs. The top-right doublet candidates get confirmed with a dedicated tool shortly

Filtering with MAD-based adaptive cutoffs

```
m.ad.hi <- function(x, k = 3) median(x) + k * m.ad(x) # m.ad() already includes 1.4826
m.ad.lo <- function(x, k = 3) median(x) - k * m.ad(x)

m.thi <- m.ad.hi(gbm$percent.mt) # actual run on this dataset = 13.1
nf.lo <- 10^m.ad.lo(log10(gbm$nFeature_RNA)) # lower bound, computed on the log scale
nc.hi <- 10^m.ad.hi(log10(gbm$nCount_RNA)) # upper bound, computed on the log scale
c(m.thi = m.thi, nf.lo = nf.lo, nc.hi = nc.hi)

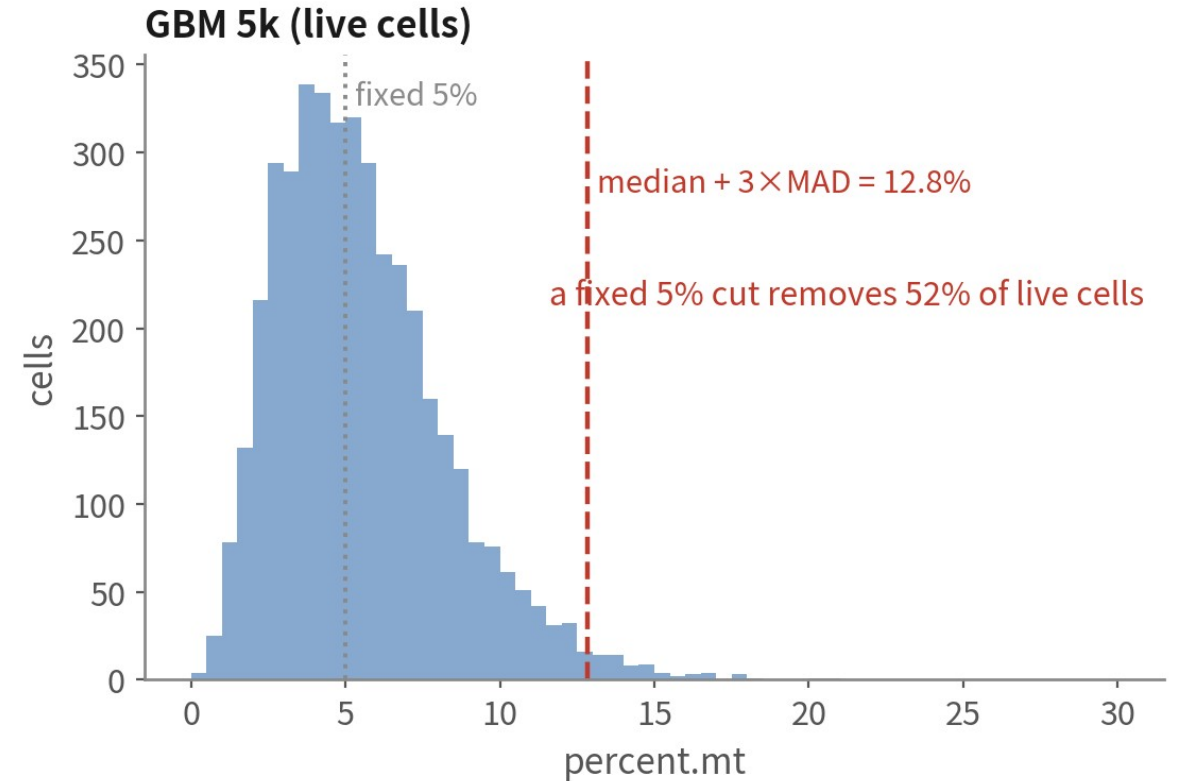
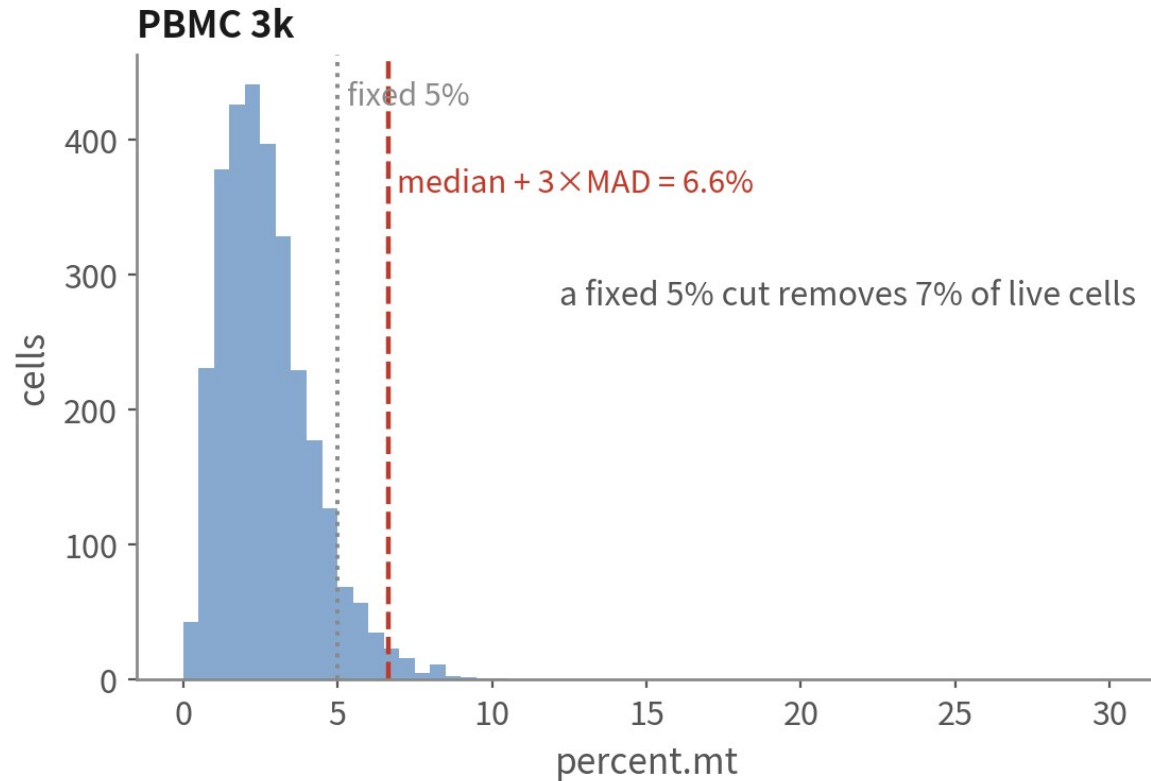
gbm <- subset(gbm, subset = percent.mt < m.thi &
              nFeature_RNA > nf.lo & nCount_RNA < nc.hi)
gbm # record how many cells were dropped
```

nCount / nFeature are usually right-skewed; on log10 they are closer to symmetric, which is what MAD needs.
percent.mt is a proportion, computed as is | Script 01 §3

Reference: Amezquita RA, et al. Nat Methods. 2020;17(2):137–145. doi:10.1038/s41592-019-0654-x

One rule, two datasets, two different lines

schematic / simulated



A copied 5% cut on GBM removes half the live cells — mostly the metabolically active malignant ones

COPY-READY METHODS SENTENCE

**"Cells were retained with:
percent.mt < median + 3×MAD (13.1%);
nFeature and nCount within median ± 3×MAD on log10 scale;
5,261 of 5,506 cells kept."**

98 of the 5,604 barcodes are dropped first by min.features = 200, so the three MAD cutoffs start from 5,506. Recompute all three on your own data.

Doublets: flag with DoubletFinder, do not delete yet

```
library(DoubletFinder)
# DoubletFinder looks for neighbours in PC space -> build a temporary object (the real clustering is in 02)
tm.p <- NormalizeData(gbm) |> FindVariableFeatures(nfeatures = 2000) |> ScaleData() |>
  RunPCA(npcs = 30) |> FindNeighbors(dims = 1:20) |> FindClusters(resolution = 0.5)
# 1. sweep pK: the only parameter we tune
bcm.vn <- find.pK(summarizeSweep(paramSweep(tm.p, PCs = 1:20, sct = FALSE), GT = FALSE))
pK.sel <- as.numeric(as.character(bcm.vn$pK[which.max(bcm.vn$BCmetric)]))
# 2. the expected doublet count is ours to supply: 10x is ~0.8% per 1,000 cells, then remove homotypic ones
nExp <- round(0.008 * ncol(tm.p) / 1000 * ncol(tm.p))
nExp <- round(nExp * (1 - modelHomotypic(tm.p$seurat_clusters)))
tm.p <- doubletFinder(tm.p, PCs = 1:20, pN = 0.25, pK = pK.sel, nExp = nExp, sct = FALSE)
pann <- grep("^pANN_", colnames(tm.p@meta.data))[1] # the column name carries the parameters, so grep it
dfcl <- grep("^DF.classifications_", colnames(tm.p@meta.data))[1]
gbm$dbl.score <- tm.p@meta.data[[pann]]
gbm$dbl.class <- tolower(tm.p@meta.data[[dfcl]])
table(gbm$dbl.class)
```

expected rate 4.2% (5,261 cells) -> nExp = 221 -> after removing homotypic = 198
the doublet count from table() equals 198 - we supplied that number, the tool did not find it

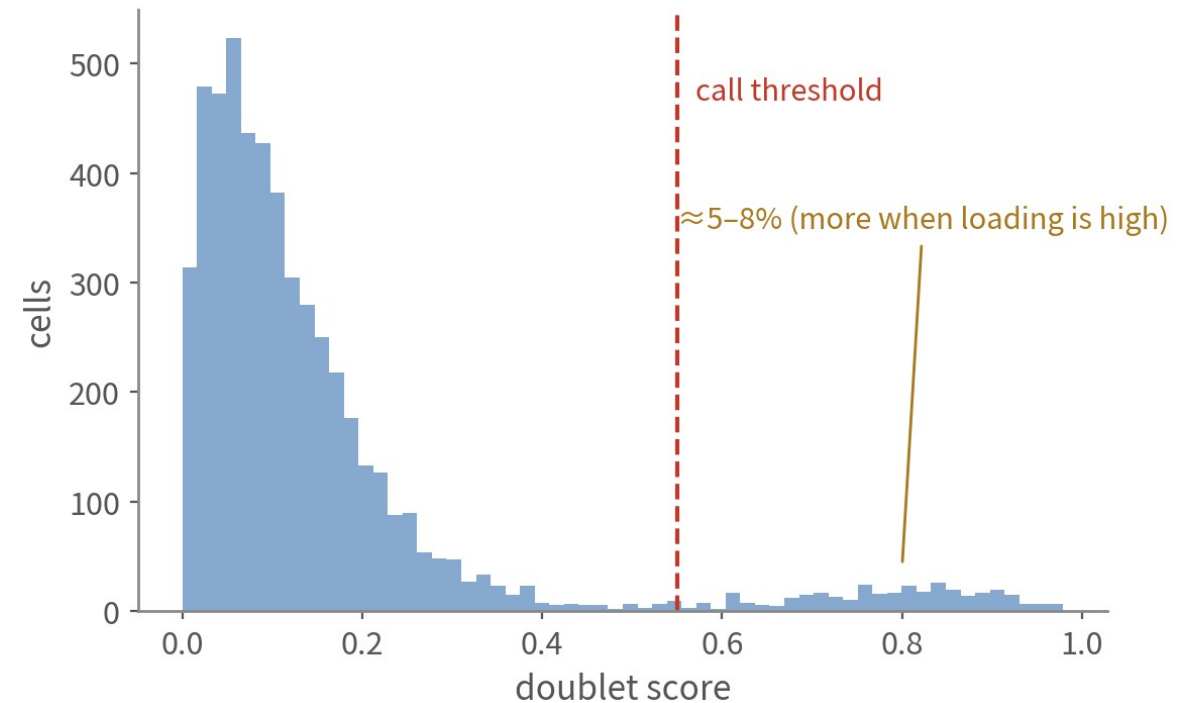
We supply the rate and the flagged count equals nExp exactly — that number belongs in our methods | Script 01 §4 | Deeper: B5

What DoubletFinder does

Doublets: the junk QC metrics miss — use a dedicated tool

schematic / simulated

- ① pick two cells at random, add their profiles → artificial doublets
- ② drop them into PC space and see which real cells sit next to them
- ③ cells surrounded by artificial doublets get a high score



Tumour doublet risk is high: a malignant cell stuck to a macrophage looks like a 'tumour cell expressing immune genes'

Choosing a doublet tool, and what rate to expect

The more cells loaded, the more doublets: the 10x rule of thumb is $\approx 0.8\%$ per 1,000 cells

Tool	Idea	Strengths	Watch out
DoubletFinder	Artificial doublets + pANN fraction; the top nExp cells are called	Published in Cell Systems; highest accuracy in Xi & Li 2021's nine-method benchmark	We supply both the expected rate and pK; the pK sweep is slow
scDbfFinder (cross-check)	Artificial doublets + kNN classifier; rate and threshold estimated automatically	One line, Bioconductor-maintained, native multi-sample	Published in F1000Research; not included in that benchmark
Scrublet (Python)	Same idea, built into scanpy	The default in Python pipelines	Nearly blind to homotypic doublets
Hashing / demuxlet	Wet-lab: antibody tags or SNPs to split samples	The only way to catch homotypic doublets; free when pooling patients	Decided at design time not afterwards

References: Xi NM, Li JJ. Cell Syst. 2021;12(2):176–194.e6. doi:10.1016/j.cels.2020.11.008 | Germain PL, Lun A, et al. F1000Res. 2021;10:979. doi:10.12688/f1000research.73600.2

03

Normalize, HVG, scale, PCA

Script 02: four lines of code, three sanity checks

Three problems, three steps

Problem 1: depth differs

Malignant cells carry more RNA than T cells: totals differ 5–10×.
Comparing raw counts compares depth

NormalizeData

divide by total $\times 1e4$, $\log_{1p}$

Problem 2: too many genes

Most of 36k genes do not vary between cells—just noise

FindVariableFeatures

keep 2,000 genes varying beyond technical expectation

Problem 3: unequal scale

MALAT1 in thousands, TFs in single digits. Unscaled, PCA is hijacked by loud genes

ScaleData

z-score per gene, clip ± 10

Every later step consumes this output: PCA runs on the scale.data of the 2,000 HVGs

A malignant cell often carries ten times a T cell's RNA — step one removes exactly that difference

Four preprocessing steps, plus cell-cycle scores

```
gbm <- NormalizeData(gbm)           # CP10K + log1p: removes depth
gbm <- FindVariableFeatures(gbm, nfeatures = 2000) # pick 2,000 HVGs
head(VariableFeatures(gbm), 10)      # take one look at the list

gbm <- CellCycleScoring(gbm, s.features = cc.genes.updated.2019$s.genes,
                        g2m.features = cc.genes.updated.2019$g2m.genes)
gbm <- ScaleData(gbm)                # z-score, clip at +/-10 (HVGs only)
gbm <- RunPCA(gbm, npcs = 50)         # compute and keep 50 PCs
```

Score the cycle now so we can see whether cycling cells form their own cluster; we do not regress by default | Script 02 §1 | Deeper: B6

LogNormalize or SCTransform?

Both are valid; this course uses the former because every step is visible and it plays well with pseudobulk and inferCNV

LogNormalize (this course)

CP10K + log1p; HVG via vst

- Simple, fast, interpretable; the data layer feeds FeaturePlot / DotPlot directly
- The counts layer stays untouched — pseudobulk, inferCNV and SoupX all need it
- With extreme depth differences (malignant vs T cells) a little depth effect remains — the PC1 check catches it

SCTransform

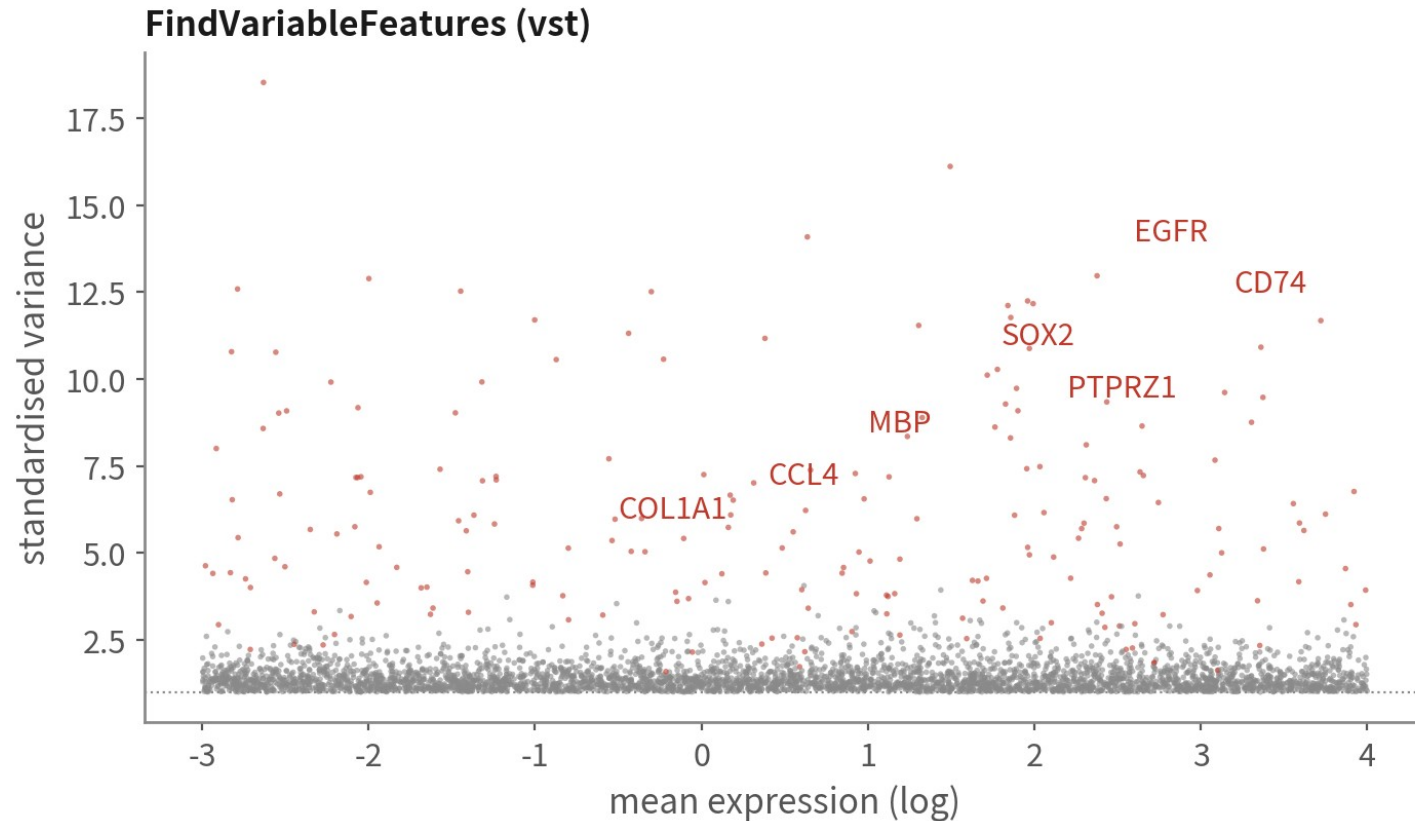
Negative-binomial regression; depth removed more cleanly

- Steadier HVGs, often cleaner clusters; v2 handles wide depth ranges well
- Adds an SCT assay; run PrepSCTFindMarkers before FindMarkers; run it per sample before integrating
- DE, CNV and pseudobulk still come from RNA counts — never from SCT values

The HVG list carries biology — but the ranking misleads

HVG: 2,000 genes with something to say, out of 36,000

schematic / simulated



Who tops the GBM HVG list

- malignant-vs-normal genes (EGFR, SOX2, PTPRZ1)
- immune genes (CD74, HLA, CCL4)
- oligodendrocyte (MBP, PLP1), vascular (VWF)
- The list itself is biology—look at it before moving on

The top 20 here are almost all stromal genes, from a 140-cell fibroblast cluster.

HVG ranks variance, not importance | Script 02 §1

Sanity check 1: PC1 is biology, not depth

```
print(gbm[["pca"]], dims = 1:2, nfeatures = 6)
cor(embeddings(gbm, "pca")[, 1], gbm$nCount_RNA) # < 0.3 fine; 0.3-0.5 check the loadings; > 0.5 investigate
```

PC_1

Positive: PM P2, CALD1, PTN, GAP43, ITM2C, SEC61G ← glial/malignant

Negative: LAPTM5, TYROBP, SRGN, FCER1G, CYBA, AIF1 ← immune

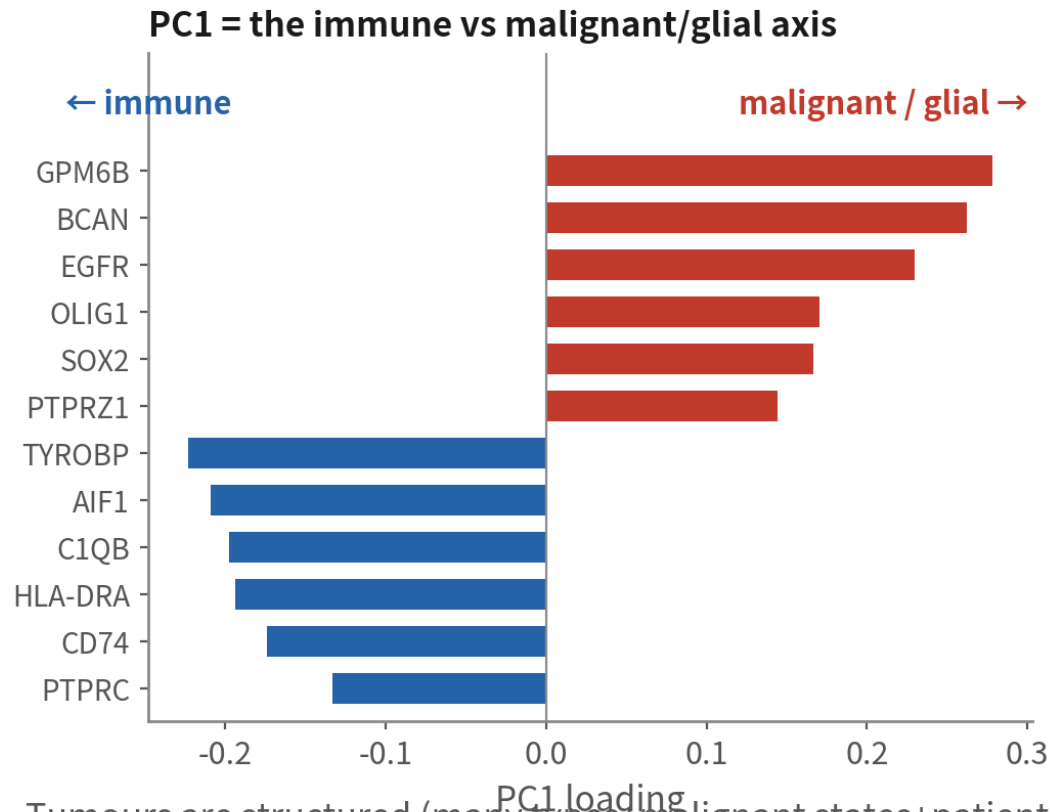
[1] 0.47

In GBM PC1 is almost always immune vs glial. 0.47 sits in the usual tumour band — judge by the loadings, not the coefficient | Script 02 §2 | Deeper: B7

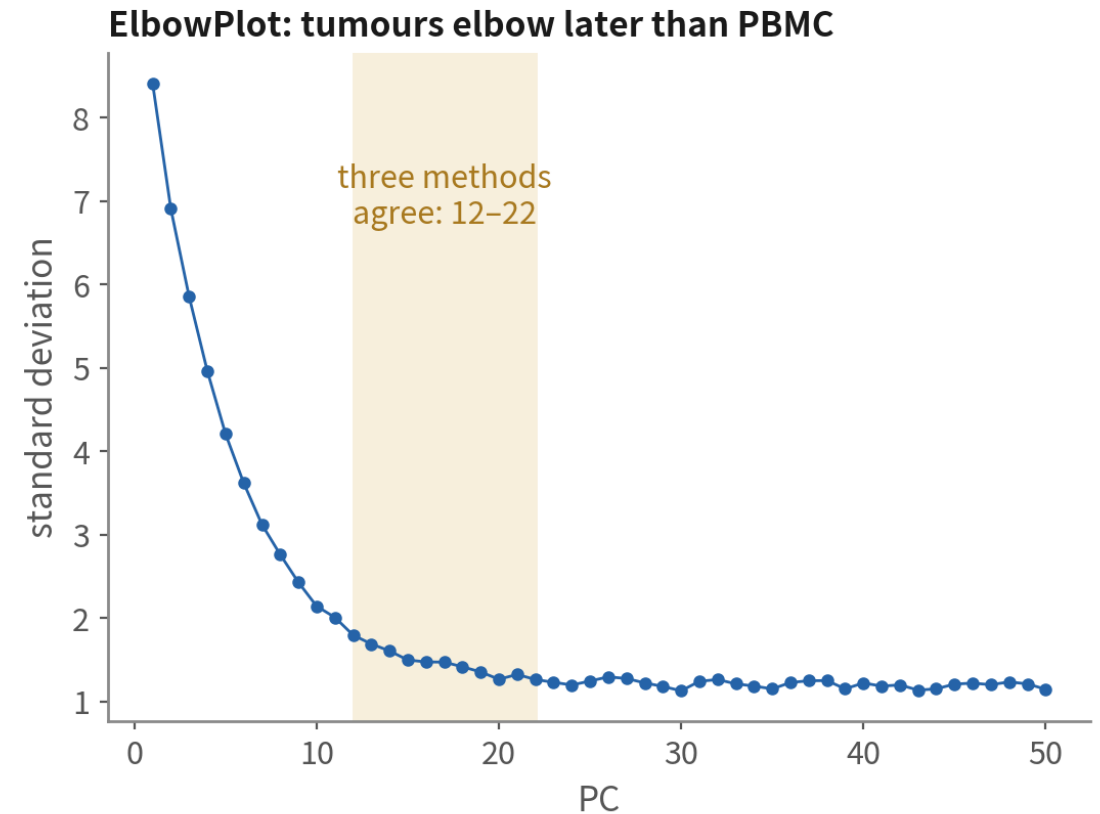
PC1 and the elbow

PCA: check PC1 is biology first, then choose how many

schematic / simulated



Tumours are structured (many types+malignant states+patient effects): use dims 20–30, more than PBMC's 10. More is safer than fewer



Tumours are structured, so the elbow comes later than PBMC. Use dims 20–30; more is safer than fewer

Sanity check 2: how many PCs

```
ElbowPlot(gbm, ndims = 50)
pct <- gbm[["pca"]]@stdev^2 / sum(gbm[["pca"]]@stdev^2) * 100
cumu <- cumsum(pct)
which(cumu > 90 & pct < 5)[1] # cumulative 90% and a single PC under 5%
npc <- 25 # GBM: the three methods agree on 20-30, so take 25
```

Turn the elbow into a number we can write in the methods. @stdev is a standard deviation: square it for variance. Too few PCs merges subtypes away, too many only adds noise | Script 02 §2

Regress out the cell cycle? Check two things first

Cycling cells in tumours are biology; do not regress by default, unless the cycle hijacks the clustering

Do not regress (default)

**Cycling cells form their own cluster;
other clusters still split by type**

- Check `DimPlot(group.by = "Phase")`: only a small S / G2M patch
- Check: no MKI67 / TOP2A among PC1–PC5 loadings
- Handling: annotate that cluster as 'cycling X', not a new type (Pitfall 3)

Regress (vars.to.regress)

**When every type is cut in
two by the cycle**

- `ScaleData(vars.to.regress = c("S.Score", "G2M.Score"))`
- Cost: the proliferation signal vanishes — often exactly what tumour studies want
- Same logic for `percent.mt`: regress only when it dominates a PC

04

Clustering and UMAP

Cluster in PC space; sweep resolution; QC again after clustering

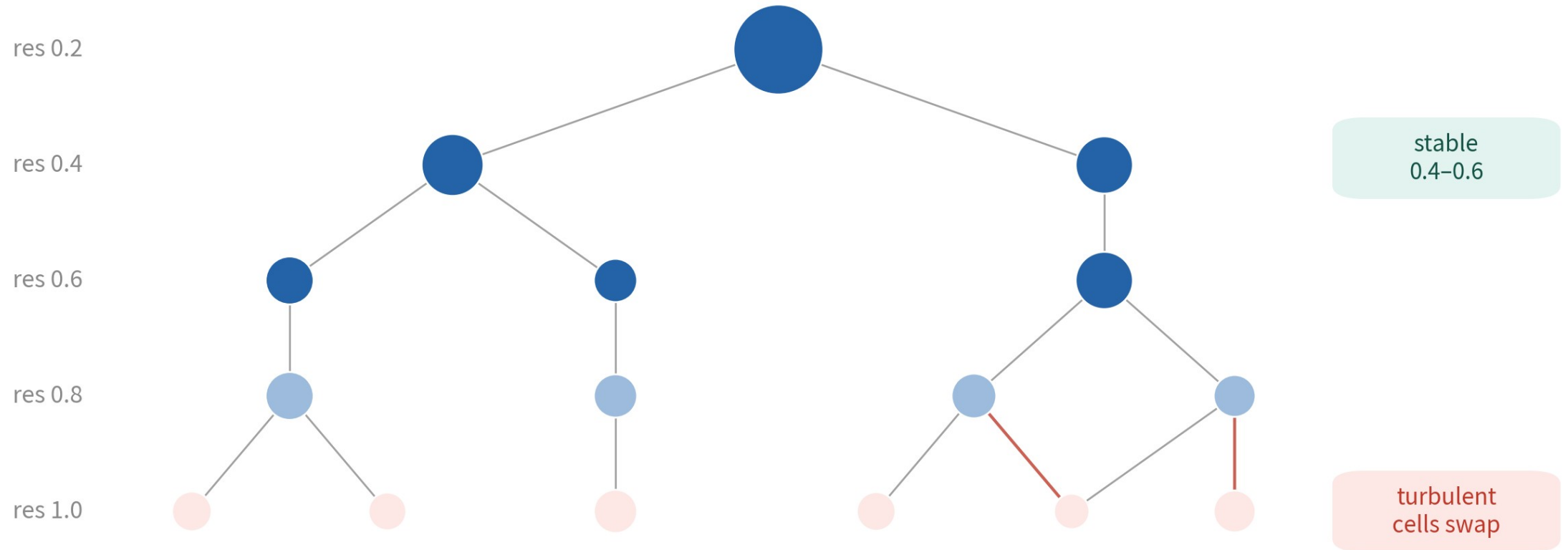
Neighbour graph → communities → projection → sweep

```
gbm <- FindNeighbors(gbm , dims = 1:npc)      # build the KNN/SNN graph in PC space
gbm <- FindClusters(gbm , resolution = 0.5)    # a starting point, not the answer
gbm <- RunUMAP(gbm , dims = 1:npc)            # the same set of dims

for (r in seq(0.2, 1.2, by = 0.2))
  gbm <- FindClusters(gbm , resolution = r, verbose = FALSE)
library(clustree); clustree(gbm , prefix = "RNA_snn_res.")
Idents(gbm ) <- "RNA_snn_res.0.5"              # after the sweep, point back to the chosen column
DimPlot(gbm , label = TRUE)
```

After the sweep the active identity is the last one run — always point it back | Script 02 §3 | Deeper: B8, B9

clustree: find the stable band



Find the band where several layers agree; step back where arrows cross and cells swap. Every extra split needs marker support

Tumour data separates immune / glial / oligo at low resolution; further cuts are malignant states and immune subsets — each needs marker support

Are the clusters stable? Change the seed, change k, watch the sizes

```
# 1. same resolution, new seed: an ARI near 1 means the clustering is stable
gbm <- FindClusters(gbm, resolution = 0.5, random.seed = 42, cluster.name = "seed42")
mclust::adjustedRandIndex(gbm$RNA_snn_res.0.5, gbm$seed42) # 0.83 here: over the line, not by much

# 2. change k.param (default 20): the skeleton of the clusters should not move
gbm <- FindNeighbors(gbm, dims = 1:20, k.param = 30)
gbm <- FindClusters(gbm, resolution = 0.5, cluster.name = "k30")
table(gbm$RNA_snn_res.0.5, gbm$k30) # a clean diagonal = stable

# 3. cluster size: treat anything under 30 cells with suspicion
sort(table(gbm$RNA_snn_res.0.5))
```

Ten seconds for three checks. ARI < 0.8, or a cluster that vanishes when k changes → an unstable cut; let markers decide whether it stays | Script 02 §3

Sanity check 3: per-cluster QC and doublet review

```
gbm @ meta.data |>
  group_by(cluster= RNA_snn_res.0.5) |>
  summarise(n = n(), m_t= median(percent_m_t), nF= median(nFeature_RNA),
            dbl= mean(dbl_class == "doublet"), cyc= mean(Phase != "G1")) |>
  arrange(desc(dbl))
```

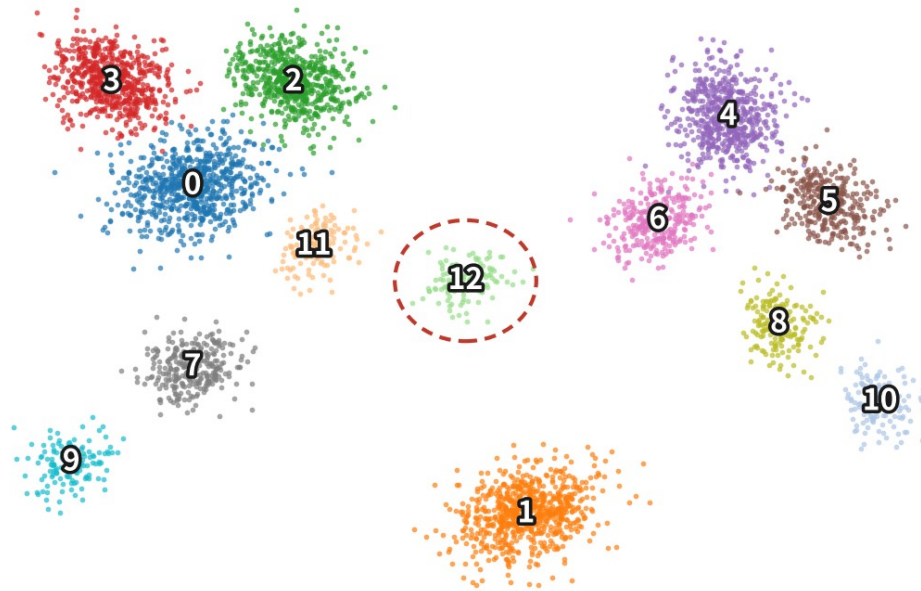
```
cluster  n   m_t  nF  dbl cyc
12      131 5.1 6221 0.60 0.24 ← 60% doublets, highest nF → drop the cluster
9       140 3.6 5370 0.31 0.20
7       276 4.4 5625 0.20 1.00 ← cyc = 1: a proliferative STATE, not a type
0      1048 4.8 4922 0.02 0.37
...
```

A cluster that is mostly doublets and sits between two big groups → remove and record; a cluster with runaway mt → back to QC | Script 02 §4

Clustering pauses here: 13 numbers, not yet 13 cell types

schematic / simulated

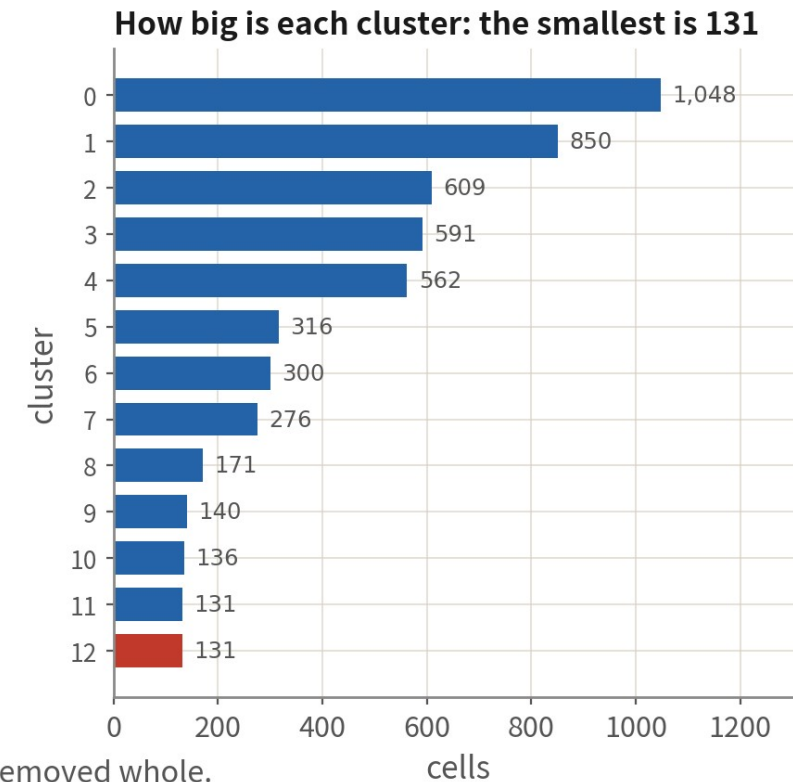
UMAP: resolution 0.5, 13 clusters (0-12)



Cluster 12 (red circle): all three doublet checks positive, removed whole

Clustering pauses here: check three is done - every cluster's QC inspected, and cluster 12 removed whole.

But we hold 13 numbers, not 13 cell types - numbers reshuffle, names do not. Naming starts in the next section.



This is what the clustering section hands over: one UMAP at res 0.5, 13 clusters, their sizes, and one cluster already flagged for removal. The numbers reshuffle with any parameter, so this is a waypoint, not a conclusion | Script 02 \$4

05

Annotation: from cluster number to cell-type name

Four slides of map first, then Script 03: gate genes → markers → SingleR → state scores

Before the methods: why annotation is hard

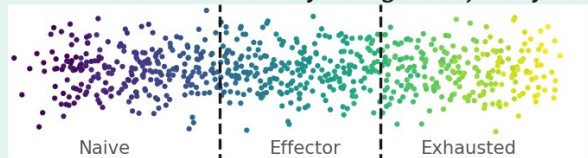
Cell annotation is not simply matching markers to labels

schematic / simulated

① Continuous biology

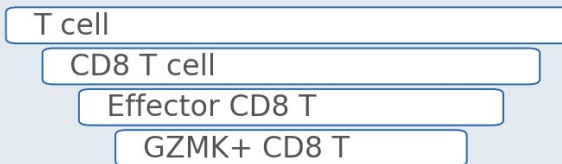
States are continuous; no natural boundary

the dashed lines are drawn by the algorithm, not by biology



② Annotation resolution

How fine depends on the question; finer is not better



③ No universal definition

Papers define and name the same population differently

one cluster



SPP1+ macrophage

TREM2+ macrophage

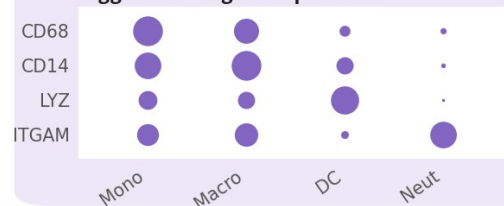
Lipid-associated Mφ

Tumor-associated Mφ

④ Non-specific markers

Markers are rarely exclusive:
read the pattern of a set

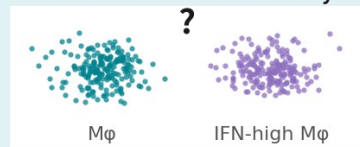
bigger dot = higher expression



⑤ Identity vs. state

A new subtype, or the same cells under IFN?

two clusters - but is that identity or state?



⑥ Technical uncertainty

Four technical sources all get misread as new populations

- ! dropout: not detected is not absent
- ! batch: a cluster can be just one batch
- ! doublet: T + B looks like a new type
- ! reference bias: missing types get forced in

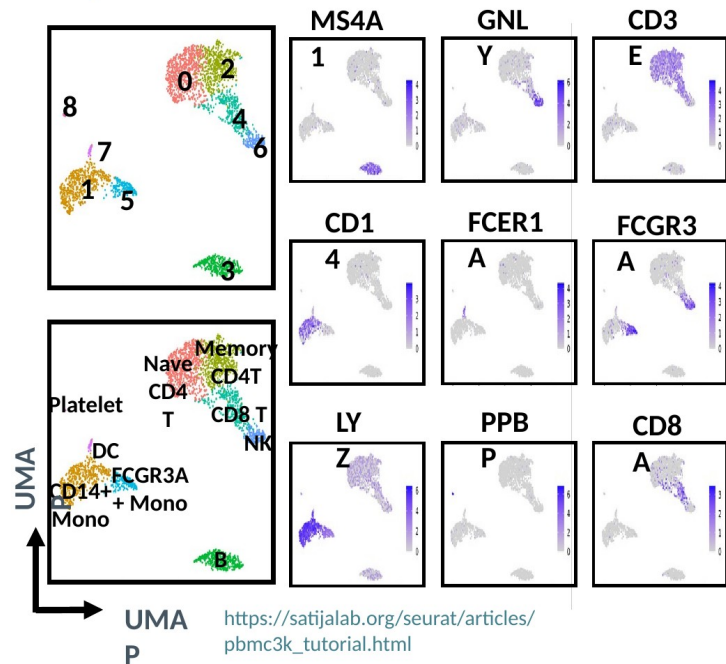
The goal is not to find the one correct answer, but to build the most reasonable, evidence-backed biological interpretation from the transcriptome, markers, reference, tissue context and the question being asked.

Not one of these six is fixed by switching tools. Knowing where the difficulty lies is what lets you see what each method solves - and what it leaves behind

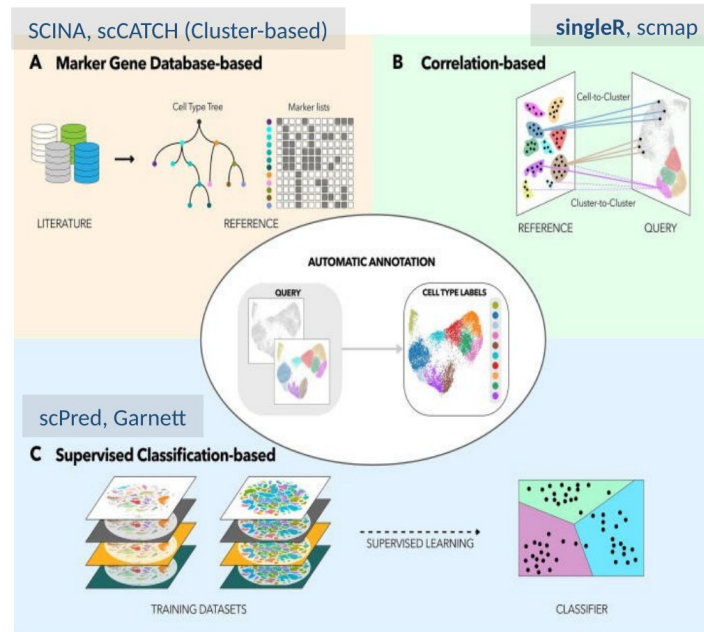
How cell types are annotated today: by hand, or automatically?

Accurate cell annotations are essential in many fields. Currently, there are many tools developed for automated annotation using reference datasets or marker genes.

Check the classic marker gene then assign by user
subjective and labor-intensive

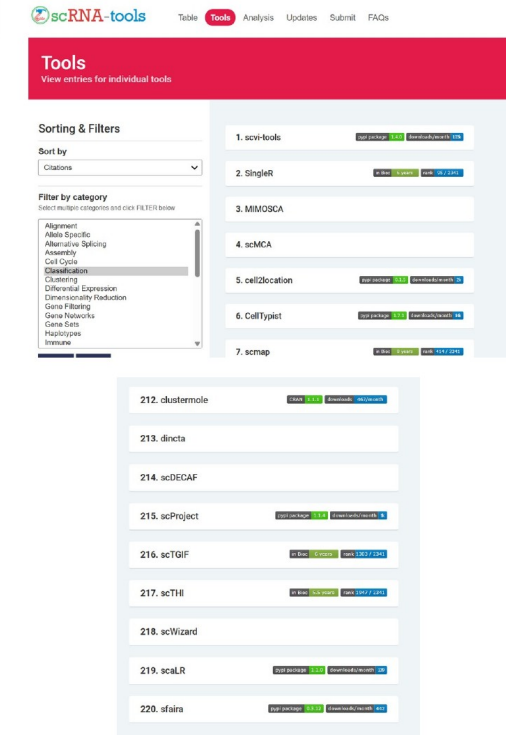


Automatic annotation Nearly 200 methods



Giovanni Pasquini, et al. Comput Struct Biotechnol J. 2021 Jan 19;19:961-969.

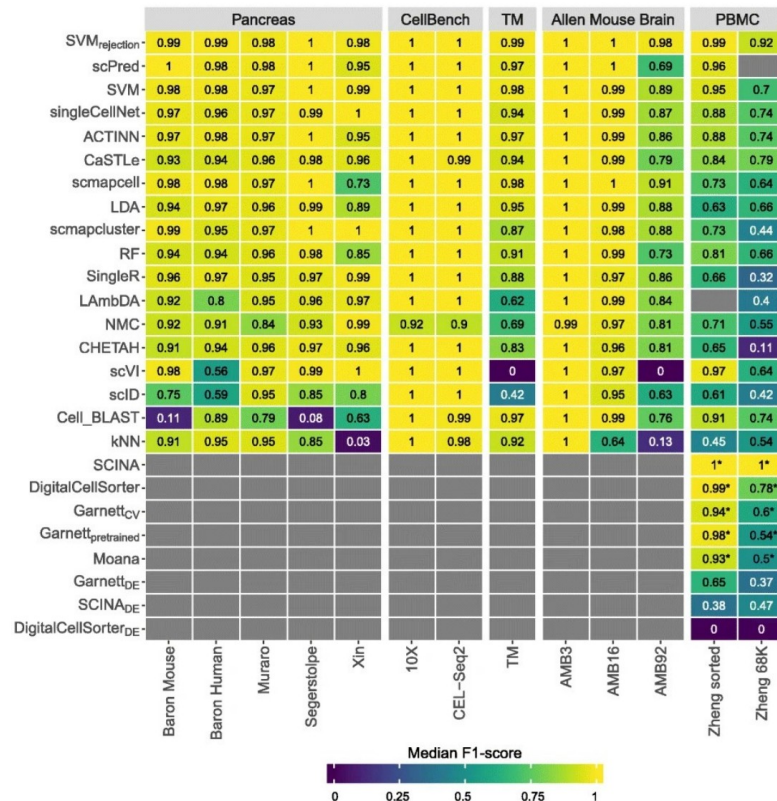
There are now more than 200 tools available for cell type annotation



<https://www.scrna-tools.org/>

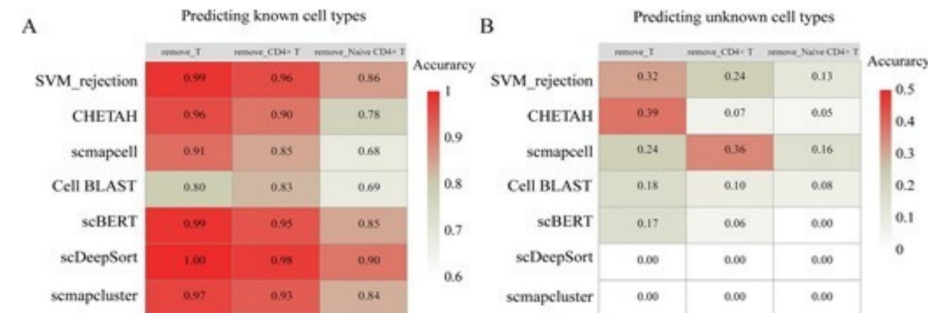
The only difference is who assigns the label. Two hundred tools, three families - and what they share matters more than what separates them: every one needs a reference first, a marker table or an annotated dataset

Robust annotation needs a good reference and a good algorithm



Abdelaal, et al. *Genome biology* 20.1 (2019): 194.

Better Reference, Better Annotation



Fu, Qiqing, et al. *Briefings in Bioinformatics* 25.5 (2024): bbae392.

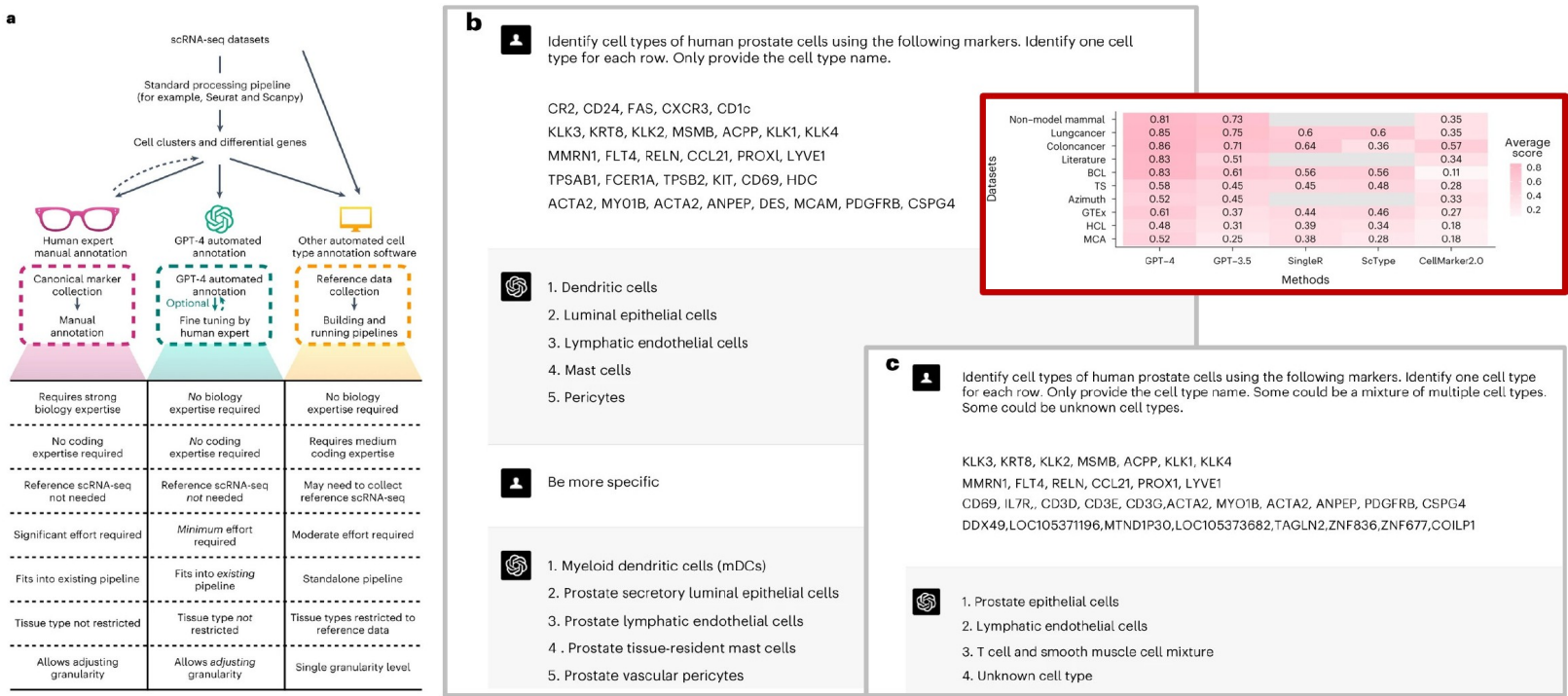
Across benchmarks and best-practice resources, the **coverage and quality of the reference atlas** often have a stronger impact on reference-based single-cell annotation than the specific classifier used.

Cheng, Jinming, et al. *BMC bioinformatics* 26.1 (2025): 22.

Swap the algorithm and the front runners barely differ; remove one type from the reference and the whole panel collapses together. The algorithm decides how well you do; the reference decides whether you can be right at all

Reading the marker list with GPT-4: where does it actually sit?

GPT-4 can accurately annotate cell types in scRNA-seq analysis based on marker genes, achieving high consistency with expert annotations across diverse tissues.



Hou, Wenpin, and Zhicheng Ji. *Nature Methods* (2024): 1-4.

Given only the marker list it beats the existing tools - but it never sees our data. It sits exactly where SingleR sits: a third line of evidence, not the only one | Script 03 §3

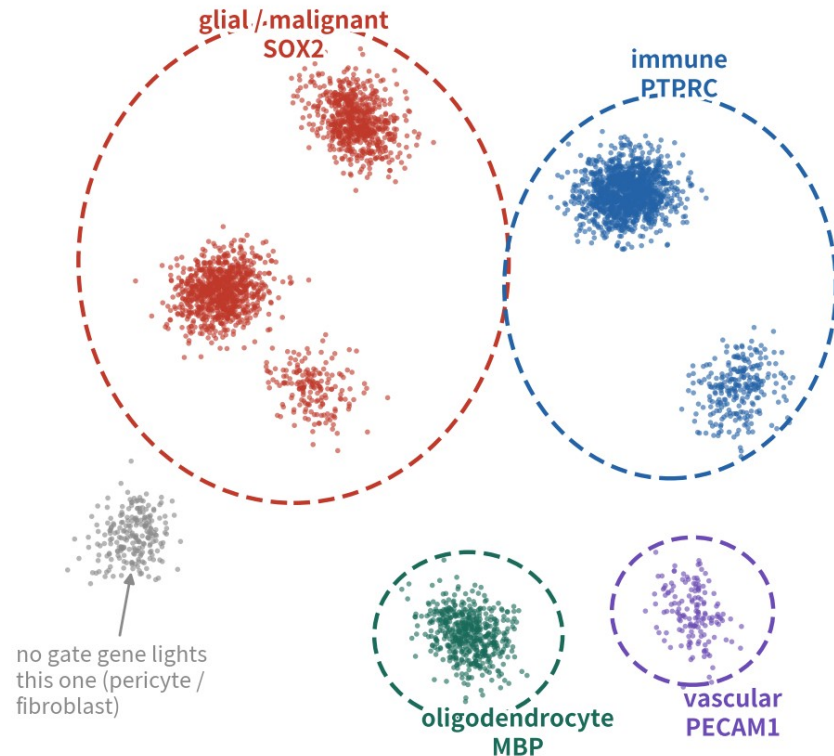
Reference: Hou W, Ji Z. *Nat Methods*. 2024;21(8):1462–1465. doi:10.1038/s41592-024-02235-4

Draw the major groups with four gate genes first

dark blue = high expression

schematic / simulated

The major groups the four gate genes carve out



PTPRC

MBP

SOX2

PECAM1

immune lights up

glial / malignant lights up

oligodendrocyte lights up

vascular lights up

Immune (PTPRC), glial/malignant (SOX2), oligodendrocyte (MBP), vascular (PECAM1).

Settle the four, then subtypes

Gate genes and the marker panel

```
FeaturePlot(gbm , features = c("PTPRC", "SOX2", "MBP", "PECAM1"), ncol= 4)

panel<- c("PTPRC", "CD68", "CD163", "P2RY12", "TMEM119", "CD3E", # immune
          "SOX2", "OLIG2", "PTPRZ1", "EGFR", # glial/malignant
          "MBP", "PLP1", "PECAM1", "VWF", "PDGFRB", "RGS5", # oligo/vascular/pericyte
          "MKI67", "TOP2A") # cycling
DotPlot(gbm , features = panel) + RotatedAxis()
```

Blocks along the diagonal mean good clusters; PTPRC lighting three immune types is expected | Script 03 §1 | Deeper: B10

A marker panel for the GBM microenvironment

population	canonical markers	one line
 immune group	PTPRC (CD45)	gate for all immune cells
 macrophage	CD68, CD163, C1QB, AIF1	tumour-associated macrophages, the most abundant immune cells in GBM
 microglia	P2RY12, TMEM119, CX3CR1	brain-resident macrophages; distinguish from blood-derived TAM
 T cell	CD3E, CD3D, CD2 (+CD8A / IL7R)	GBM is immune-cold; T cells are usually scarce
 malignant / glial	SOX2, OLIG2, PTPRZ1, EGFR	markers only say 'glial lineage'; malignancy needs CNV (Q3)
 oligodendrocyte	MBP, PLP1, MOG	invaded normal white matter; a good inferCNV reference
 endothelial	PECAM1, VWF, CLDN5	prominent vascular proliferation in GBM
 pericyte	PDGFRB, RGS5, ACTA2	perivascular stromal cells
 cycling	MKI67, TOP2A	a state, not a type; overlays malignant cells

Microglia vs blood-derived macrophages, and cycling state vs cell type — the two pairs most often confused in tumour annotation

Evidence 1: the FindAllMarkers table

```
m arkers <- FindAllMarkers(gbm , only.pos = TRUE,  
                           min.pct = 0.25, logfc.threshold = 0.5)  
m arkers |> group_by(cluster) |> slice_max(avg_log2FC, n = 5) |>  
  select(cluster, gene, avg_log2FC, pct.1, pct.2) |> print(n = 50)
```

cluster	gene	avg_log2FC	pct.1	pct.2	
4	OLR1	3.65	0.98	0.17	← macrophage (TAM)
1	MAG	7.64	0.94	0.03	← oligodendrocyte: pct.2 = 0.03, clean
0	PTPRZ1	1.92	0.99	0.38	← glial (undetermined): high pct.2 = shared lineage

Read in order: avg_log2FC → pct.1 / pct.2 → p. pct.2 near 0 is the clean marker | Script 03 §2

Filtering the marker table: four columns, four questions

Look at p last — with 5,000 cells nearly everything is significant

Column	It answers	A good marker	Trap
avg_log2FC	How much higher than the rest	> 1 (over two-fold)	Low genes get their FC squashed by the pseudocount
pct.1	Fraction expressing inside the cluster	> 0.5	A 'marker' at pct.1 = 0.3 describes part of the cluster — a hidden subset or doublets
pct.2	Fraction expressing elsewhere	< 0.1	Ambient RNA inflates pct.2; MBP at pct.2 = 0.3 does not mean oligodendrocytes everywhere
p_val_adj	Whether the difference is chance	< 0.05 (almost all are)	Cells in a cluster are not independent; p is overstated by orders of magnitude — use only as a sieve

Evidence 2: SingleR automated annotation

```
library(SingleR); library(cellidex)
ref <- cellidex:HumanPrimaryCellAtlasData()
pred <- SingleR(test = GetAssayData(gbm, layer = "data"), ref = ref,
               labels = ref$label.main, clusters = gbm$seurat_clusters)
data.frame(cluster = rownames(pred), SingleR = pred$labels,
           delta = round(pred$delta.next, 2))
```

cluster	SingleR	delta
0	Astrocyte	0.12 ← no GBM malignant in the reference — forced to the nearest
1	Astrocyte	0.08 ← actually oligodendrocyte (MAG/HAPLN2); HPCA has none either
4	Macrophage	0.18
9	Tissue_stem_cells	0.05 ← actually pericyte / fibroblast
10	T_cells	0.06 ← immune types are where it is genuinely accurate

Immune types are accurate; anything absent from the reference (malignant, oligodendrocyte) gets forced into the nearest normal type — a reminder, not an answer | Script 03 §3

Reference: Aran D, et al. Nat Immunol. 2019;20(2):163–172. doi:10.1038/s41590-018-0276-y

Where automatic annotation stands (tumour data as the example)

Automatic annotation (SingleR /Azimuth) on tumour data

- ✓ Immune cells: accurate—references (HPCA, Blueprint) cover them well
- ✓ Oligodendrocytes, endothelium: accurate
- ✗ Malignant cells: no 'GBM malignant' in any reference—forced onto the nearest normal type (astrocyte, neural progenitor)
- Verdict: use it as a second line of evidence; call malignancy separately

Cross-check table (example)

cl.	manual	SingleR	verdict
2	macrophage	Macrophage	✓
3	T cell	T_cells	✓
4	oligo	Oligodendrocyte	✓
0	malig. AC-like	Astrocyte	CNV decides
1	malig. OPC-like	Neural progenitor	CNV decides
5	endothelial	Endothelial	✓

Three-column cross-check: agreeing clusters pass; for the glial clusters both columns stop at lineage — this dataset cannot call malignancy

Annotation strategies: four roads, usually walked together

No single road is reliable alone; this course goes gate → markers → reference → scores. All four stop at lineage — malignancy needs a different class of evidence

Strategy	How	Strength	Weakness	Tools
Manual markers	Dotplot / FeaturePlot per cluster against literature markers	Interpretable; finds novel types	Slow, subjective, cluster-dependent	Seurat
Reference set	Correlate each cell / cluster with an annotated reference	Fast, reproducible; accurate for normal types	Types absent from the reference (tumour) get forced into the nearest	SingleR, scType, CellTypist
Gene-set scoring	Mean / AUC of a gene set as a score	Continuous, comparable suits states	High score ≠ type; thresholds are subjective	AddModuleScore, UCell, AUCell
Atlas mapping	Project cells onto a large atlas and inherit labels	Finest subsets, consistent labels	Needs a good atlas malignant cells cannot be mapped	Azimuth, scArches

Choosing an automatic annotation tool

Run all three on the same GBM data: normal types agree above 90%, and the disagreements are exactly where a human has to look

Tool	Idea	Reference	Unit	Best for / watch out
SingleR	Spearman correlation + fine-tuning	celldex (HPCA, Blueprint, Monaco immune)	Cell or cluster	Default for normal types; an immune reference splits T subsets
scType	Marker database + cluster-level scoring	Built-in human / mouse tissue markers	Cluster	Results in seconds; brain table has no tumour
CellTypist	Logistic-regression models	Immune atlas, multiple human tissues	Cell	Strongest on immune subsets (Python)
Azimuth	Anchor mapping onto a reference UMAP	PBMC, brain, lung kidney, marrow...	Cell	Comes with UMAP coordinates; the brain reference does not apply to GBM malignant cells
scANVI / scArches	Semi-supervised deep model	Any atlas we train	Cell	Cross-technology, large scale; needs GPU

References: Abdelaal T, et al. Genome Biol. 2019;20(1):194. doi:10.1186/s13059-019-1795-z | Hao Y, et al. Cell. 2021;184(13):3573–3587.e29. doi:10.1016/j.cell.2021.04.048

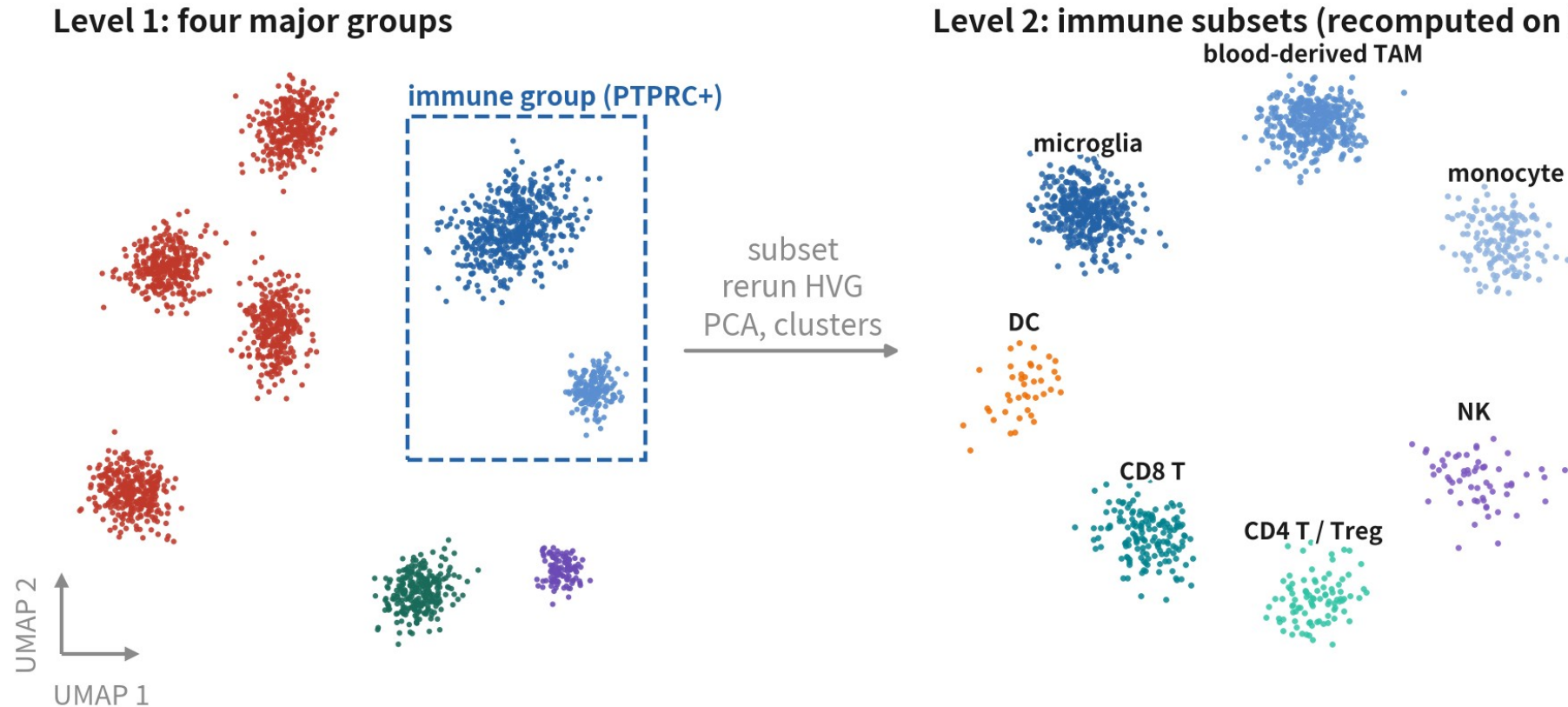
Naming: the label goes exactly as far as the evidence

```
new.ids <- c("0" = "G lial (undetermined)", "1" = "O ligodendrocyte",  
            "2" = "G lial (undetermined)", "3" = "G lial (undetermined)",  
            "4" = "Macrophage", "5" = "Macrophage", "6" = "Macrophage",  
            "7" = "Cycling (undetermined)", "8" = "M icroglia",  
            "9" = "Pericyte / fibroblast", "10" = "T cell",  
            "11" = "G lial (undetermined)", "12" = "G lial (undetermined)")  
# which cluster to drop is decided by the QC table from 02 - never hard-code the number  
qc <- read.csv("output/tables/02_per_cluster_qc.csv")  
new.ids[as.character(qc$cluster[qc$dbl > 0.5])] <- "DOUBLET"  
gbm <- RenameIds(gbm, new.ids); gbm$celltype <- Idents(gbm)  
gbm <- subset(gbm, subset = celltype != "DOUBLET")
```

Number-to-name maps differ for every dataset — check our own DotPlot first; which cluster to drop comes from 02's QC table, never a hard-coded number | Script 03 §4

Hierarchical annotation: major groups settled, now the subsets

schematic / simulated



Why recompute

- 1 Global HVGs are dominated by malignant-vs-immune; within-immune differences never make the top 2,000
- 2 Same for PCs—subset signal sits beyond PC 20
- 3 Rerun on immune cells alone and CD4/CD8, microglia vs blood-derived TAM separate

Take the immune major group out and rerun HVG, PCA and clustering — only then do CD4 / CD8 and microglia / blood-derived macrophages separate. Malignant 'subsets' use state scores, not clusters

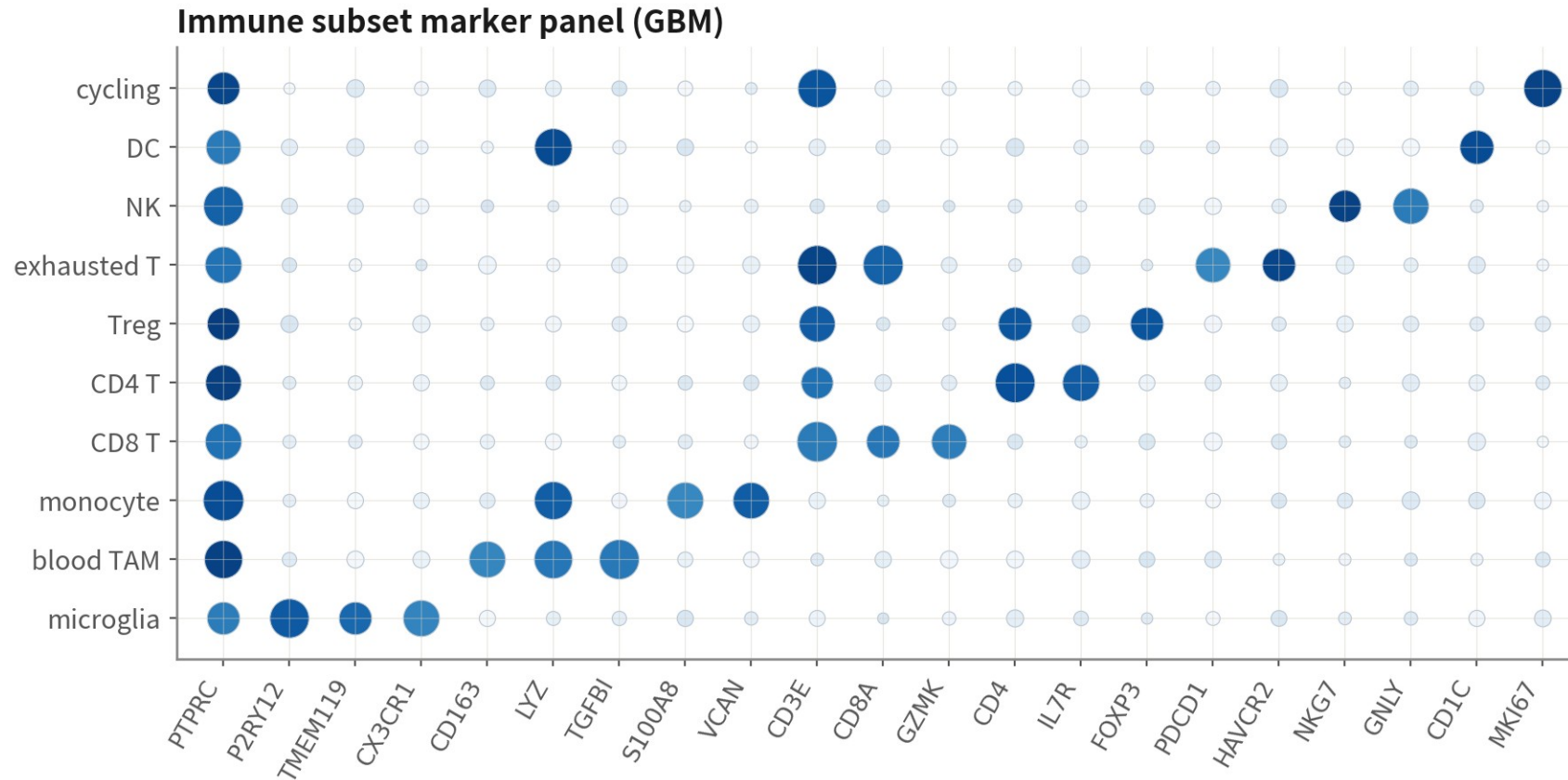
Subsets: subset, then rerun the whole pipeline

```
in_m <- subset(gbm, celltype %in% c("Macrophage", "Microglia", "T cell"))
in_m <- NormalizeData(in_m) |> FindVariableFeatures(nfeatures = 2000) |>
  ScaleData() |> RunPCA(npcs = 30)
ElbowPlot(in_m) # a subset usually needs only 15-20 PCs
in_m <- FindNeighbors(in_m, dims = 1:20) |> FindClusters(resolution = 0.6) |>
  RunUMAP(dims = 1:20)
in_m.panel <- c("P2RY12", "TMEM119", "CX3CR1", # microglia
  "CD163", "LYZ", "TGFB1", "S100A8", "VCAN", # blood-derived TAM / monocyte
  "CD3E", "CD8A", "GZMK", "CD4", "IL7R", "FOXP3",
  "PDCD1", "HAVCR2", "NKG7", "GNLY", "CD1C", "MKI67")
DotPlot(in_m, features = in_m.panel) + RotatedAxis()
# to split one cluster without rerunning everything: FindSubCluster(gbm, cluster = "2", graph.name = "RNA_snn")
```

Always recompute HVGs and PCA after subsetting; circling on the old UMAP is a common error. Do not split subsets under ~200 cells | Script 03 §4b

The immune subset panel: the diagonal is the evidence

schematic / simulated



How to read

PTPRC lights the whole column: a gate, not a subset marker

Microglia vs blood TAM: P2RY12/TMEM119 vs CD163/TGFB1

Exhausted T=CD8 +PDCD1/HAVCR2, not a new type

Cycling immune lights only MKI67: a state—merge back

Microglia vs blood-derived TAM, CD8 vs exhausted CD8, Treg — every cut needs its own markers; a group lit only by MKI67 is a state, merge it back

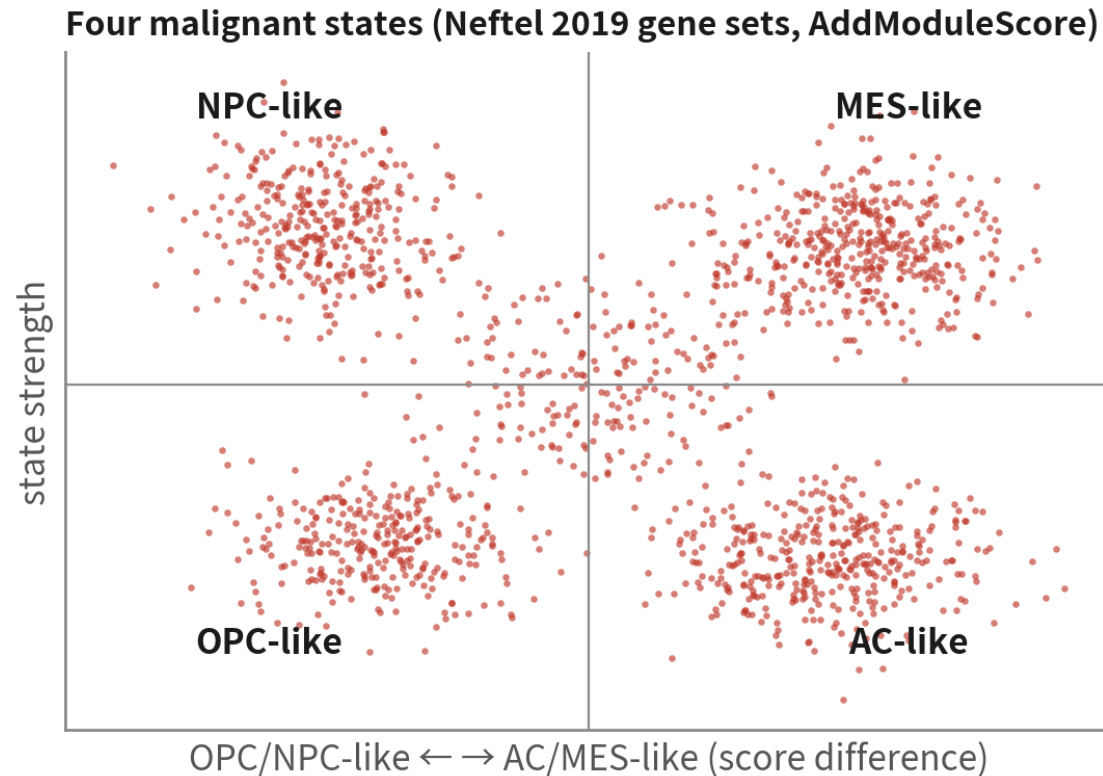
State scores for the glial clusters (Neftel 2019)

```
neftel<-list(                                # the first six of each set, as a demo
  MES = c("CHIBL1", "ANXA2", "ANXA1", "CD44", "VIM", "MT2A"),
  AC  = c("CST3", "S100B", "SLC1A3", "HOPX", "MT3", "SPARC1"),
  OPC = c("BCAN", "PLP1", "GPR17", "FBN1", "LHFPL3", "OLIG1"),
  NPC = c("DLL3", "DLL1", "SOX4", "TUBB3", "HES6", "TAGLN3"))
glial<-subset(gbm, idents = "Glial(undetermined)")
glial<-AddModuleScore(glial, features = neftel, name = names(neftel))
FeaturePlot(glial, features = paste0(names(neftel), 1:4), ncol = 4)
```

Six per set here; use the paper's supplementary table for the full sets (50 each). The states were defined inside confirmed-malignant cells — a score is never evidence of malignancy | Script 03 §5 | Deeper: B13

Why score states rather than cluster again

schematic / simulated



- Malignant states are continuous; hard clusters invent boundaries
- Scoring with published gene sets makes results comparable across studies
- One patient's tumour holds all four states at once—one source of therapy resistance
- Score cell cycle (G2M/S) too, or cycling cells form their own cluster

Malignant states are continuous; scores keep that continuity and are comparable across studies. Read the cycle score alongside

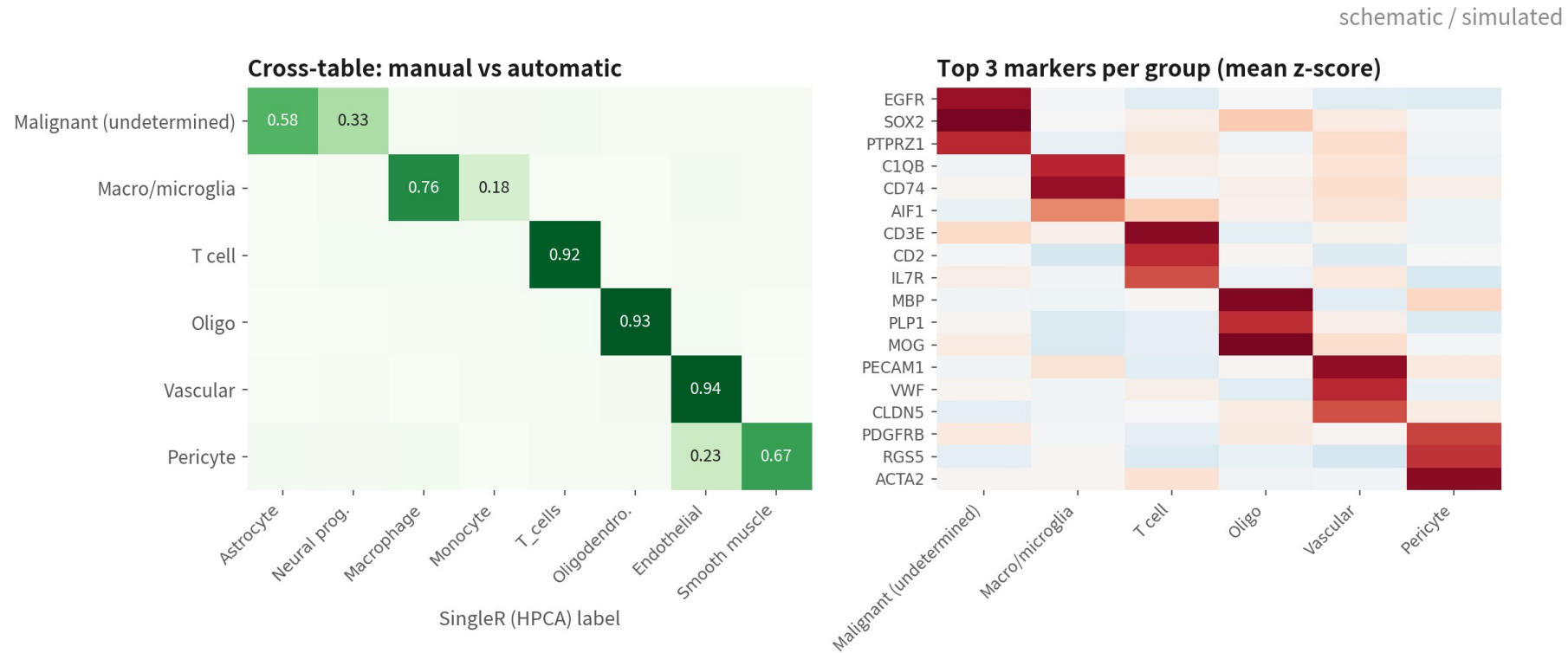
Three ways to score: AddModuleScore, UCell, AUCell

```
# 1. AddModuleScore (Seurat): mean of the set minus the mean of control genes at similar expression.  
# Fast, but sensitive to sequencing depth  
glial <- AddModuleScore(glial, features = nefte1, name = names(nefte1))  
  
# 2. UCell: a Mann-Whitney U score from each gene's rank inside the cell; robust to depth and sparsity  
library(UCell)  
glial <- AddModuleScore_UCell(glial, features = nefte1, name = "_UCell")  
  
# 3. AUCell: how many set members fall in the top 5% of ranked genes per cell (AUC); SCENIC uses it too  
library(AUCell)  
rank <- AUCell_buildRankings(LayerData(glial, layer = "counts"))  
auc <- AUCell_calcAUC(nefte1, rank)  
  
FeaturePlot(glial, features = paste0(names(nefte1), "_UCell"), ncol = 4)  
cor(glial$MES1, glial$MES_UCell) # AddModuleScore appends an index, UCell does not; ranks agree
```

Report both the algorithm and the source of the gene set; when the three disagree, trust UCell, which is least sensitive to depth | Script 03 §5

References: Andreatta M, Carmona SJ. Comput Struct Biotechnol J. 2021;19:3796–3798. doi:10.1016/j.csbj.2021.06.043 | Aibar S, et al. Nat Methods. 2017;14(11):1083–1086. doi:10.1038/nmeth.4463

Annotation quality control: the cross-table and the marker heatmap



Left: a clean diagonal; malignant splitting into astrocyte/neural progenitor is expected (no tumour in the reference). Right: blocks along the diagonal, no group lighting everything

Left: a clean manual-vs-automatic diagonal with every deviation explained; right: top markers light in blocks, no group lights everything. Both go into the supplement

QC code: cross-table, heatmap, cluster tree, silhouette

```
# cross-table: manual labels vs SingleR (each row sums to 1)
round(prop.table(table(gbm$celltype, gbm$singleR), 1), 2)

# top-5 marker heatmap per cluster (at most 100 cells each, or the figure gets too heavy)
top5 <- markers %>% group_by(cluster) %>% slice_max(avg_log2FC, n = 5)
gbm <- ScaleData(gbm, features = unique(c(VariableFeatures(gbm), top5$gene))) # scale heatmap genes first
DoHeatmap(subset(gbm, downsample = 100), features = top5$gene, group.by = "celltype")

# cluster tree: is the similarity structure sensible (immune on one branch, glia on another)
gbm <- BuildClusterTree(gbm, dims = 1:20); PlotClusterTree(gbm)

# silhouette: how self-contained each cluster is in PC space (under 0.1 is suspect)
library(cluster)
cl <- if(ncol(gbm) > 20000) sample(colnames(gbm), 20000) else colnames(gbm) # dist() is O(n^2)
sil <- silhouette(as.integer(Ids(gbm)[cl]), dist(Embeddings(gbm, "pca")[cl, 1:20]))
apply(sil[, "sil_width"], Ids(gbm)[cl], mean)
```

Four checks, four questions: does the automatic label agree, are markers clean, is the structure sensible, are clusters separable | Script 03 §5b

Naming convention: keep three label levels in metadata

Papers use level 2, supplementary tables level 3, statistical tests often only level 1

Column	Content	Example	Rule
celltype_l1	Major group	Immune / Malignant / Oligo / Vascular / Stromal	Five to seven; composition analysis uses this
celltype_l2	Type	Microglia, blood-derived TAM CD8 T, glial (undetermined)	Each backed by ≥ 2 markers; names aligned to Cell Ontology
celltype_l3	State / subset	Exhausted CD8 T, cycling TAM, MES-like	States as 'adjective + type'; do not coin new nouns
celltype_conf	Confidence	high / medium / unresolved	Write unresolved when unsure; never force

Common mis-annotations: five 'novel types' and what they really are

Looks like	Actually	How to catch it	What to do
Tumour cells expressing immune genes	Malignant + macrophage doublets	High per-cluster doublet rate, high nCount, sits between two big groups	Remove the cluster and record it
A stem-like subset	A cycling existing type	Markers are only MKI67 / TOP2A; Phase nearly all S/G2M	Merge back, label 'cycling'
A stress-response subset	Dissociation artefact	FOS / JUN / HSPA1A high; a slice inside every type	Regress or annotate; not biology
Undifferentiated / unknown type	Low-quality cells	Lowest nFeature, highest mt, no marker lights	Back to QC
Immune cells expressing oligodendrocyte genes	Ambient RNA	MBP / PLP1 a little in every group high pct.2	Rerun after SoupX

References: van den Brink SC, et al. Nat Methods. 2017;14(10):935–936. doi:10.1038/nmeth.4437 | McGinnis CS, Murrow LM, Gartner ZJ. Cell Syst. 2019;8(4):329–337.e4. doi:10.1016/j.cels.2019.03.003

Deliverables: the three plots and one table of a Figure 1

```
p1 <- DimPlot(gbm, group.by = "celltype", label= TRUE,
             repel= TRUE) + NoLegend()
p2 <- DotPlot(gbm, features = panel, group.by = "celltype") + RotatedAxis()
p3 <- ViolinPlot(gbm, features = c("nFeature_RNA", "percent.mt"),
               group.by = "celltype", pt.size = 0, ncol= 2)
fig1 <- (p1 | p2) / p3 # patchwork layout
ggsave("output/figs/03_fig1_annotation.pdf", fig1, width = 14, height = 9, bg = "white")

comp <- gbm @ meta.data %>% dplyr::count(celltype) %>%
  mutate(pct= round(100 * n / sum(n), 1))
write.csv(comp, "output/tables/03_composition.csv", row.names = FALSE)
```

celltype	n	pct
Glia (undetermined)	2379	46.4
Macrophage	1178	23.0
Oligodendrocyte	850	16.6
Cycling (undetermined)	276	5.4
Microglia	171	3.3
Pericyte / fibroblast	140	2.7
T cell	136	2.7

UMAP for position, dotplot for evidence, violin for QC consistency, the table for numbers. Hand in all four and half the first-round questions disappear | Script 03 §6

Wrap-up: save the object, record the versions

```
saveRDS(gbm, "output/rds/03_gbm_annotated.rds")  
write.csv(markers, "output/tables/03_markers.csv", row.names = FALSE)  
sessionInfo() # goes in the supplement; a reviewer's best friend
```

Every file in output/ can be regenerated from the scripts in R/ — that is what reproducible means | Script 03 §6

06

COMMON PITFALLS

The six most common hands-on traps in tumour data

Pitfall 1: copying the 5% mito cut



What it is

Applying the PBMC tutorial's 5% to GBM.

Why it happens

Every tutorial uses PBMC; 5% looks like common sense.

What it costs you

Half the live cells gone, mostly active malignant ones.
Immune proportions get inflated and every downstream composition analysis is skewed.

Pitfall 2: a doublet cluster reported as 'immune-like tumour cells'



What it is

A cluster expressing both SOX2 and CD74 written up as an 'antigen-presenting malignant subset'.

Why it happens

The markers look exciting and there is literature to cite.

What it costs you

Sixty percent of the cluster is flagged by the doublet tool and it sits between malignant and myeloid. Sanity check 3 exists for exactly this.

Pitfall 3: cycling cells reported as a new type



What it is

An MKI67/TOP2A-high cluster named a 'stem-like subpopulation'.

Why it happens

Cell-cycle signal is strong and easily dominates clustering.

What it costs you

It is a state, not a type: dividing malignant cells and macrophages mixed together. Check the Phase column first, then decide whether to regress.

Pitfall 4: cranking the resolution up and naming every cluster



What it is

Turning the resolution up to 1.2, getting twenty-odd clusters, and giving every one a name.

Why it happens

More clusters look more refined, and clustree still draws something.

What it costs you

The extra cuts have no marker support and vanish when the seed changes. The number of clusters is not the goal; being able to justify each cut with markers is. Go back to the stable band, then let markers decide whether to split further.

Pitfall 5: reading the marker table by p value alone



What it is

FindAllMarkers returns $p_{adj} = 1e-40$, so that gene is taken as the cluster's marker.

Why it happens

A tiny p value looks like strong evidence.

What it costs you

A cell-level test treats cells as samples, so with thousands of cells the p values are tiny by construction and rank poorly. Read pct.1 against pct.2 and the size of avg_log2FC as well — a gene at $pct.1 = 0.22$ versus $pct.2 = 0.19$ separates nothing, however small its p_{adj} .

Pitfall 6: copying someone else's marker panel



What it is

Lifting a PBMC tutorial's marker panel wholesale onto tumour data.

Why it happens

The panel is ready-made and looks generic.

What it costs you

The immune half still works, but the tissue-specific half is simply missing — without PTPRZ1, MBP and PECAM1, the glial, oligodendrocyte and vascular cells all fall into unknown. A marker panel is rebuilt for the tissue and the cancer type, not copied.

Q2 checklist



raw object saved
before filtering



each QC cutoff has a
number and a reason (MAD)



doublet scores
stored in metadata



$|\text{cor}(\text{PC1}, \text{nCount})| < 0.3$



dims justified (elbow
+quantified cutoff)



resolution swept; stable
band backed by markers



four major groups
settled with gate genes



every name has two
independent lines of
evidence (markers+automatic)



malignant label provisional:
'glial (undetermined)'

All nine boxes before the object goes to Q3

ONE-LINE SUMMARY

**Every parameter is a hypothesis: a number, a reason, a record.
Annotation settles lineage first; malignancy awaits
multiple lines of evidence.**

Do this and our pipeline is reproducible and defensible. Calling malignancy needs evidence one sample cannot give; Q3 switches datasets to show it.

Check yourself 1

GBM: median percent.mt 7%, median + 3×MAD = 13%.

Where does the cutoff go?

- A The conventional 5%
- B 13%, with the number and reason in the methods
- C Skip the percent.mt filter

Answer B. A threshold is a property of the data. 5% removes most live cells; no filter keeps the dying-cell mode.

Check yourself 2

A cluster expresses both SOX2 and C1QB; the doublet tool flags 65% of it. What now?

- A An exciting new subset — report it
- B Remove the whole cluster and record why
- C Remove only the flagged 65%

Answer B. Mostly flagged and sitting between two big groups — the whole cluster is a doublet product; the remaining 35% are the ones the tool missed, not real cells.

Check yourself 3

SingleR labels cluster 0 as Astrocyte; markers are SOX2 / PTPRZ1 / EGFR. The honest label?

A Astrocyte

B Malignant

C Glial (malignancy undetermined)

Answer C. No GBM malignant cells exist in the reference, so SingleR picks the nearest; markers only give lineage. Malignancy needs genome-level evidence plus patient-specificity — one patient cannot supply both, so undetermined is the honest label.

Check yourself 4

clustree shows res 0.4, 0.5 and 0.6 agreeing, and cells start swapping clusters from 0.8 up. Which do you take?

- A 0.8 — more clusters, finer detail
- B 0.5 — the middle of the stable band, then check each cut with markers
- C Run them all and keep whichever looks best

Answer B. A stable band means those cuts survive a change of parameter, and the middle of it is the safest pick. Choosing the prettiest result means letting the answer decide the method — any line can be justified after the fact.

Check yourself 5

A marker has $\text{padj} = 1\text{e-}40$ but $\text{pct.1} = 0.22$ and $\text{pct.2} = 0.19$. Can it be used?

- A Yes — a padj that small must be real
- B No — nearly the same fraction of cells express it inside and outside
- C It depends on whether avg_log2FC is positive or negative

Answer B. A cell-level test treats cells as samples, so with thousands of cells padj is tiny by construction. A 0.03 gap between pct.1 and pct.2 means the gene is just as common outside the cluster as inside — no discrimination, no marker.

Check yourself 6

One cluster lights up none of the four gate genes. What is the reasonable next step?

- A Label it unknown and move on
- B Delete it — we cannot identify it anyway
- C Read its own top markers and QC first, and subset-rerun if needed

Answer C. The gate genes only cover the four major groups, so pericytes and fibroblasts are expected to fall outside. Read the cluster's own top markers first — they are often recognisable at a glance — and check its QC, because low-quality cells also light up nothing. Once it is not junk, subset and rerun.

NEXT

Q3: multi-sample integration and downstream analysis

Four patients, core vs periphery (GSE84465) — the dataset changes precisely to supply what one sample cannot.

Malignancy is called from multiple lines of evidence led by inferCNV, pseudobulk puts DE at the patient level, and the episode closes with a downstream routing guide.

